

THE NEW/STACK

Kubernetes at the Edge

Container Orchestration at Scale

Charles Humble

sponsored by



The New Stack

Kubernetes at the Edge: Container Orchestration at Scale

Alex Williams, Founder and Publisher

Ebook Team:

Ben Kubany, Project Manager

Charles Humble, Author

Diana Gonçalves, Graphic Designer

Michael Baker, Copy Editor

Susan Hall, Copy Editor

Vicki Walker, Ebook Editor

Supporting Team:

Andrea Washington, Account Director

Andrew Tillery, Senior Marketing Manager

Heather Joslyn, Editor-in-Chief

Laila Bailey, Email Marketing Manager

Michelle Maher, Copy Editor

Shelley Chen, Social Media Manager

Tony Sala, Director of Sales

© 2025 The New Stack. All rights reserved.

20250917.1

Table of Contents

Sponsor5

The Many Meanings of Edge Computing: Defining the Edge.....6

 What — and Where — Is the Edge?6

 Advantages of the Edge9

 Downsides of the Edge..... 12

 From Docker to Edge Orchestration 14

Edge Computing in Action: Real-World Applications Across Industries 17

 Agriculture 18

 Energy and Renewables22

 Healthcare26

 Retail..... 31

The Edge Vendor Landscape: Evaluating Your Options 34

 Key Decision Factors for Economic Buyers35

 Market Survey: Leading Kubernetes Edge Vendors.....37

 Rakuten Cloud Native Platform37

 Red Hat Device Edge38

 Scale Computing HyperCore 40

 Sidero Talos Linux and Omni..... 43

 Spectro Cloud Palette 48

 SUSE Edge Suite 51

 How To Choose the Right Edge Kubernetes Platform53

Kubernetes at the Edge: Applying Tips and Tricks54

 The Edge of Disaster55

 How To Assemble a Team56

 Gradually Raise the Stakes.....57

 Prioritize Day 2 Operations.....59

 The Edge of Tomorrow 61

Final Words62

 Other Resources 64

About the Author65

Disclosure..... 66

Sponsor

We are grateful for the support of our sponsor for this ebook:



Sidero Labs, creator of Talos Linux and Omni, simplifies and secures Kubernetes across bare metal, data center, edge, and cloud.

CHAPTER 01

The Many Meanings of Edge Computing: Defining the Edge

What — and Where — Is the Edge?

Like many IT terms, edge computing can mean multiple things, which can make it complicated to discuss. No matter the definition however, “the edge” refers to a location — physical, logical, or both — outside the core IT infrastructure.

Traditionally, core infrastructure meant an on-premises data center housing an organization’s computer servers and data storage. Over time, this definition has grown to encompass not only on-premises corporate data centers, but also colocation facilities, as well as cloud data centers run by hyperscalers such as Amazon Web Services (AWS), Google Cloud Platform (GCP), or Microsoft Azure.

Even with edge computing, the architectural principles remain the same: Core infrastructure refers to an organization’s IT foundations and includes the elements essential to its business operations. It encompasses the technology needed to run and manage the organization, such as databases, operating systems, networks, and hardware. And it is distinct from the core logic or business rules of the business itself.

In a corporate setting, edge might refer to users who are off-premises, accessing the core data center through a virtual private network (VPN) or internet connection. In this context, the edge refers to a place on the network topology.

However, the edge also refers to the location of computing and storage; put simply, the edge is a place, not just a reference to the topology. This location definition is further subdivided into three distinct tiers: near edge, far edge, and device edge:

Near edge typically refers to an edge data center, that is to say a small,

decentralized computing facility located close to end users and devices, designed to process data locally to reduce latency and improve performance for real-time applications. For example, for a corporate user:

- The off-premises user connects to the edge data center over the internet or through a VPN.
- Then the edge data center connects with the core data center through a wide area network (WAN), typically a software-defined WAN (SD-WAN).

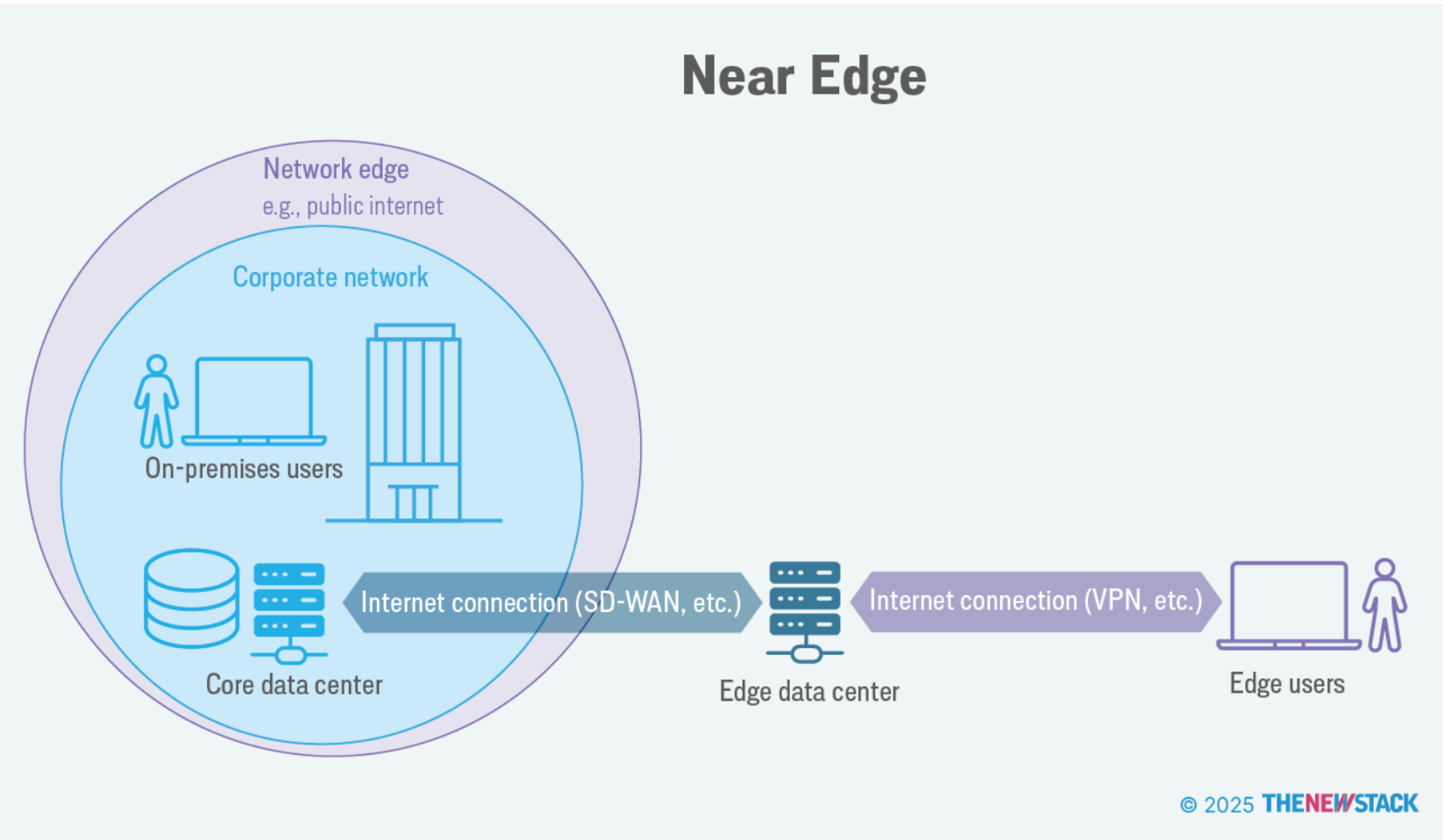


FIG 1.1 Near-edge architecture

Near-edge locations can have their own content delivery network (CDN) caches, security frameworks, and performance-enhancing mechanisms.

Far edge represents the most decentralized locations, with infrastructure placed in immediate proximity to the end devices and users, such as at the base of a cell tower or point-of-sale systems in retail or restaurants.

Device edge refers to the location of endpoints such as sensors and controllers with specialized functions; it encompasses the actual devices, called edge devices, and their immediate

vicinity. The devices connect to the corporate data center or cloud location either via a cell tower or an edge server. These connected edge devices, which might include smart cameras, industrial sensors, wearable health monitors, and medical imaging scanners, span a wide range of industries, from retail and transportation to healthcare and manufacturing.

Edge devices are typically dedicated to one specific purpose, such as video surveillance of a particular area, or controlling some type of machinery, such as an automated workstation on an assembly line. A typical edge device will have just enough computing capacity to

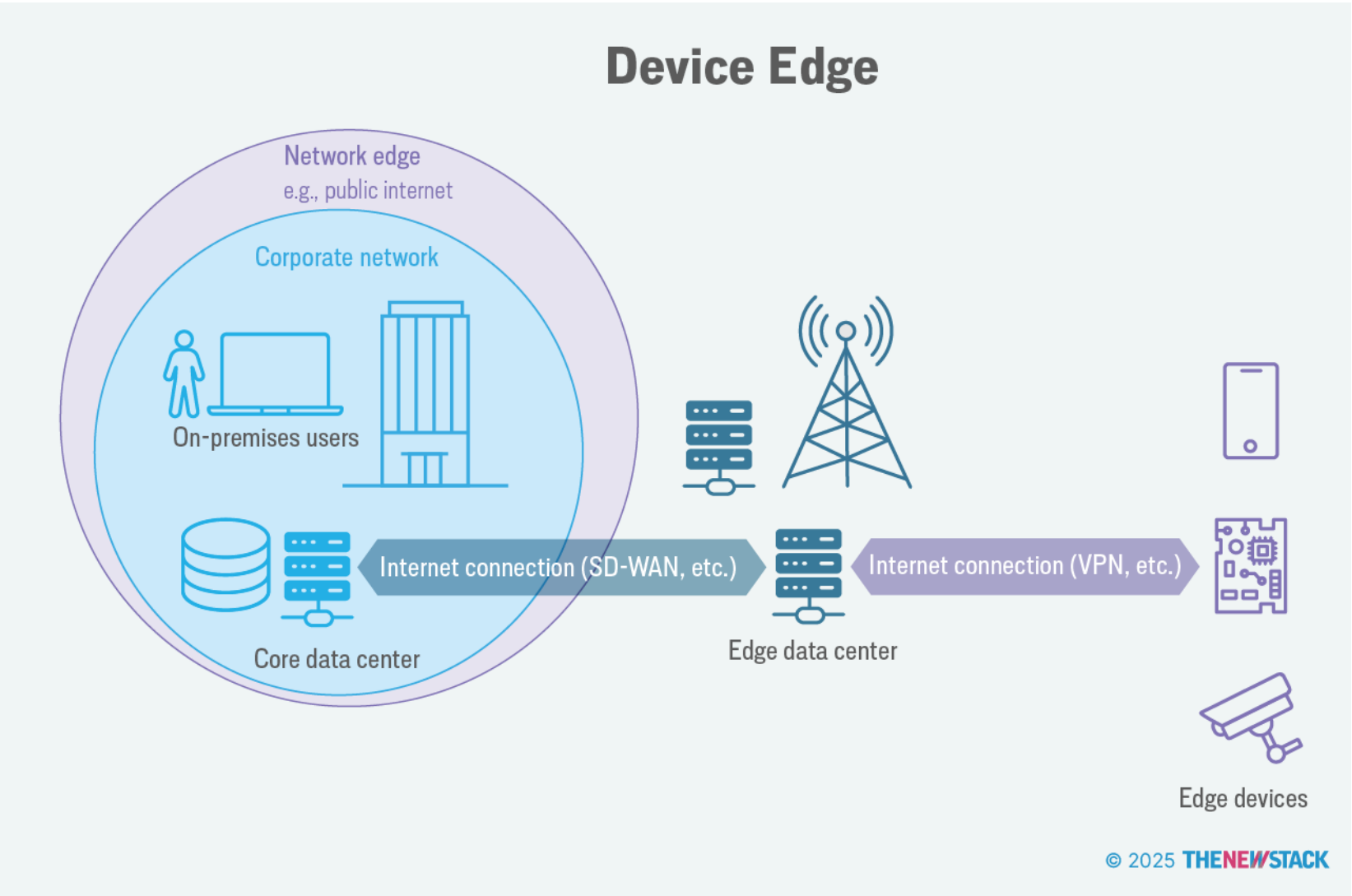


FIG 1.2 Device edge architecture

Advantages of the Edge

support its dedicated activities. Edge devices are also what we mean when we talk about the Internet of Things (IoT), whether that is consumer or industrial IoT.

Another way to think about this is that edge computing spans multiple tiers — from small data centers and regional facilities (near edge), to retail stores and restaurants (far edge), down to individual appliances (device edge) — each requiring different approaches based on connectivity, resources, and the criticality of the workloads.

“Centralization also comes at a cost to privacy and, depending on the nature of the data to be processed, can fall foul of data sovereignty regulations.

Both near and far edge have six primary advantages over the centralized model:

1. Reduced latency and faster response times by processing data closer to the source.
2. Improved network bandwidth efficiency by reducing the need to send large amounts of data to the cloud.
3. Lower carbon footprint through minimization of network traffic.
4. Enhanced data security and privacy by processing sensitive information locally.
5. Increased reliability and resilience due to a decentralized architecture.
6. Better support for real-time applications and AI and machine learning (ML) models.

To understand this better, consider the device edge. Regardless of whether these devices are in remote wind farms,

factories, hospitals, or vehicles, modern embedded computing devices generate vast quantities of data. As an example, a modern wind turbine might have sensors to monitor vibrations, temperature, and sound, along with lightning loggers and acoustic sensors that use AI to detect damage or predict failures.

Because they generate so much data, edge devices are a logical target for AI and ML. As with many general edge use cases, the traditional approach to data analysis and ML requires sending data to a centralized location for processing.

This allows the organization to use high-performance hardware (such as graphical processing units, or GPUs) and offers the scope for a data science team to perform in-depth data analysis to improve model performance.

Centralizing data, however, poses considerable challenges, such as intermittent and unreliable connectivity, constrained bandwidth, and high latency. In our wind turbine example, a lack of bandwidth might well mean it is not practical to send all the data generated back to the core data center.

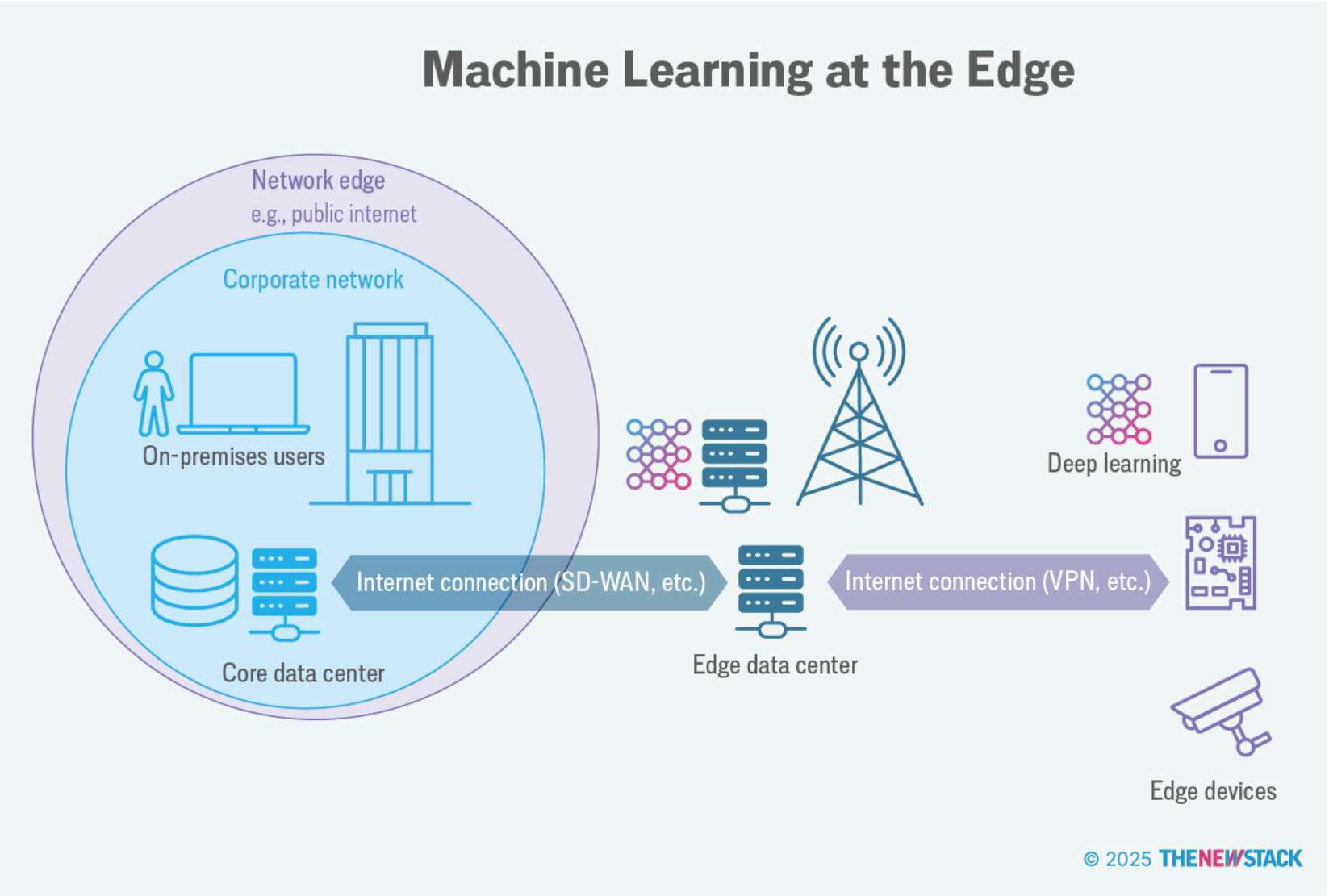


FIG 1.3 Architecture of machine learning at the edge.

Centralization also comes at a cost to privacy and, depending on the nature of the data to be processed, can fall foul of data sovereignty regulations.

Given these factors, it makes sense to shift processing of that data closer to where it is being generated, as this can provide a number of advantages. Moreover, if the requirement is to train a machine learning model, doing so at the edge can be better from a privacy point of view. By combining on-device federated learning with other techniques, such as differential privacy and secure aggregation to limit data leakage, you can strengthen privacy.

The edge isn't just about AI, however. Green energy producer Vandebron, based in the Netherlands, manages a large and varied edge environment that includes hundreds of wind turbines and solar farms. The Vandebron team constantly tracks the energy generated at these locations across the Netherlands to balance the grid and match demand at any given time. The firm provides an online marketplace for

sustainable energy, directly connecting supply and demand for individual and business customers. Its marketplace enables buyers to know exactly where their energy comes from and where their money goes.

More broadly, if the goal is to build more sustainable or greener IT systems, running more calculations at the edge minimizes network traffic, which, as I explained in [“The Developer’s Guide to Cloud Infrastructure, Efficiency, and Sustainability,”](#) accounts for roughly half of the IT industry’s carbon emissions. It also allows you to take advantage of energy stored in batteries and potentially apply time-based [demand shifting](#) to charge the devices at times when the energy is cleanest.

Moreover, minimizing the number of on-demand processes running in data centers reduces the need for cooling, which uses a substantial amount of energy and creates corresponding emissions in centralized data centers. As with any data center, edge data centers do require cooling, of course,

Downsides of the Edge

but by locating these smaller data centers closer to where people live, the waste heat can be repurposed to heat homes and other buildings or municipal swimming pools. [T.Loop](#) is one example of a data center company that transforms energy waste into heat.

When the goal is to reduce costs, cloud demigration — moving workloads from hyperscale cloud providers back to edge locations — is increasingly common. Demigration can also reduce latency and improve resource utilization while maintaining the ability to burst to the cloud from the data center when needed.

These advantages — better privacy, lower carbon emissions, and lower costs — are important, but given that everything in computing is a series of trade-offs, it should be no surprise that running computer systems at the edge also comes with challenges. The two key ones are:

- **Limited computing resources:** Edge devices are getting more capable. However, size, weight, and power (SWaP) constraints mean that devices deployed to the edge are limited in terms of CPU/GPU power and memory.
- **Intermittent or denied network connectivity:** Many computing capabilities are designed for “always-connected” networks and do not handle intermittent connections gracefully. Software written for tactical edge nodes must be robust to operate in areas of limited network infrastructure or where communications are actively denied because of adversary efforts or natural disasters.

Writing for Carnegie Mellon University's Software Engineering Institute, [Marc Novakouski and Grace Lewis](#) describe two types of systems that are particularly sensitive to these challenges:

- The **humanitarian edge**, where first responders and other emergency personnel are “operating in unsafe conditions caused by weather events, seismic events, or even infrastructure failures, in which things can change at a moment’s notice.”
- The **tactical edge** is the field environment where “military personnel execute missions (i.e., ‘boots on the ground’) and can range from air to ground to undersea, and from urban to rural settings.”

To bring these capabilities to the edge, Novakouski and Lewis suggest that software engineers must design for “distributability” — the ability to split system components across multiple

computing nodes similar to the way you would for a set of microservices running in a data center.

Your circumstances may be less extreme than these situations, but similar challenges — from limited hardware and poor connectivity to bad actors, such as [Russia’s Sandworm group](#) or [China’s Volt Typhoon](#) — still apply.

It is also worth emphasizing that, very often, users at the edge are not technologists. They might be medical professionals entering patient medical data or someone managing the electric vehicle (EV) charging station outside a grocery store. This has implications: You have to lock down devices so that users don’t inadvertently delete data, make sure devices are properly hardened from a security standpoint, and make updates as easy as possible to deploy.

These challenges help explain why containerization is becoming an increasingly popular method for deploying software at the edge.

From Docker to Edge Orchestration

Carnegie Mellon University has a long association with edge devices ever since its computer science students created the first IoT device. In 1982, they connected a vending machine to the ARPANET — an early precursor to the internet — so they could check whether it was empty without leaving their lab.

In 1991, to save people working in the University of Cambridge, England, computer laboratory the disappointment of finding the nearby coffee maker empty, the first webcam was set up, providing a live picture of the coffee pot to all desktop computers on the office network.

While the concept of containerization has existed since Unix's chroot system was introduced in 1979, it has grown in popularity since Docker's release in 2013. Containers are now ubiquitous — used across all areas of software and instrumental for the CI/CD pipeline in many projects.

Containerization is a virtualization method where an application's software files (including code, dependencies, and configuration files) are bundled into a package and executed on a host by a **container runtime engine**. The package is called a **container image**, which then becomes a **container** when it is executed. While similar to virtual machines (VMs), containers do not virtualize the operating system kernel (usually Linux) and instead use the host's kernel.

Removing some of the resource overhead associated with virtualization makes containers lightweight, which makes them a good choice for constrained edge devices. However, it also has some security ramifications since containers are less isolated,

as well as less portable, than VMs. This means the operating system must be hardened to limit the means of access.

Most modern container engines use the [Open Container Initiative](#) (OCI) image format. OCI images are a representation of a container and the software that should run within it, making it possible to create new containers in a predictable, repeatable way.

Having a standard packaging format provides an additional element that is equally valuable: **immutable infrastructure**. In an immutable system, a completely new image is built, and the update replaces the original in a single operation. There are no incremental updates or changes. Additionally, building a new image means the old image is still around and can be quickly used for a rollback if something goes wrong.

To explain further, while working for a UK retail bank in the early 2000s, I spent many painful hours installing software on the four clustered Windows servers that were to run the then-new

internet banking system. The machines needed to be identical, down to the exact order in which software and patches were installed, since one piece of software would replace older binaries and make incremental updates to configuration files. Individual configuration files also had to be updated by hand. The process was absurdly difficult and error-prone, to the extent that we rarely had all four machines in the cluster available at the same time.

Immutable infrastructure seemed a radical idea a decade ago. But because it eliminated so many profoundly frustrating deployment issues, it quickly became a standard. With device edge, where you might be deploying to hundreds, thousands, or tens of thousands of devices, this predictability is essential.

At scale, however, automating the deployment, management, scaling, and networking of containers becomes burdensome. Thankfully, container engines provide a standard interface for controlling running containers.

This means that third-party tools can stop and start containerized applications or change the resources assigned to them. Typically, container management is handled by a container orchestrator — a piece of software written expressly for this purpose.

Kubernetes is long established as the [most widely adopted approach](#) for orchestrating containers on a large scale, but adopting it at the edge is relatively new. Doing so maintains the

familiar operational models from data centers, a distinct advantage for busy IT teams.

It’s worthwhile to note that different edge layers require tailored solutions, with full Kubernetes suitable for near-edge and far-edge deployments, lightweight Kubernetes distributions optimized for resource-constrained far edge environments, and containerization tools like [Podman](#) more commonly used for device edge endpoints.



SUMMARY

Founded in 2019, Sidero Labs, Inc., the creator of Talos Linux and Omni, focuses on bringing simplicity and security to bare-metal and edge Kubernetes. Together, these tools are trusted by hundreds of companies and help manage tens of thousands of clusters worldwide.

KEY FEATURES

- By delivering scalable API-driven management for Kubernetes clusters in any environment, Talos Linux and Omni are making on-prem infrastructures secure by default, easier to use, and more reliable to operate.
- Talos Linux is a minimal, immutable, and API-managed operating system designed specifically for running Kubernetes.
- Omni is a SaaS platform that enables enterprise Kubernetes management across bare metal, data centers, cloud, and edge environments.

CHAPTER 02

Edge Computing in Action: Real-World Applications Across Industries

Back in 2018, analysts JM Chabas, Chandra Gnanasambandam, Sanchi Gupte, and Mitra Mahdavian from McKinsey & Company identified [more than 100 potential edge computing use cases](#). Since then, more capable edge hardware, networking improvements, and progress in multiple areas of AI/ML have opened even more potential uses. This means that while some of McKinsey's use cases remain more theoretical than practical, edge computing now encompasses a vast array of industries.

Some real-world examples across four industries — agriculture, energy, healthcare, and retail — illustrate what edge computing is being used for, with a particular focus on applications where Kubernetes and containers are part of the mix.

Agriculture

Agriculture is a hugely important potential application for edge and AI technologies.

The United Nations believes that the global [human population will peak at 10 billion to 11 billion in the 2080s](#) — a considerable increase from the estimated [8 billion](#) on the planet in 2022. As the population continues to grow, farmers are under pressure to produce food with less reliance on fossil fuels, pesticides, and herbicides. And they are expected to do this while climate change exacerbates extreme weather events, degrades the already limited arable land, and increases water scarcity.

While crop yields in most parts of the world have improved dramatically, in sub-Saharan Africa they've remained low. In "[Not the End of the World](#)," [Hannah Ritchie](#) reported that "the average cereal yield in Africa is about half that of India, and about one-fifth of the yield of the U.S." Africa will need to grow a lot more food in the coming decades, Ritchie states: "In the next 30

years, it is expected to add another billion to its population, and then another billion in the 30 years after that."

The techniques that have worked well elsewhere — fertilizers, improved seed varieties, and irrigation — will likely be a major part of the puzzle. Another promising technique is precision agriculture, which applies advanced technologies such as AI, robotics, and drones to optimize crop and meat production while minimizing environmental impact.

However, the cost barriers put precision agriculture beyond the reach of the small and midsize farmers responsible for much of the world's food supply. These farms may also lack power or internet connectivity due to their geographic isolation, and farmers may not be IT savvy.

In 2023, a cross-company team produced an architecture for an [intelligent edge for sustainable agriculture](#). This project, which won [TM](#)

[Forum's Outstanding Catalyst](#) award in the Sustainability and Impact on Society category, produced a reference architecture blueprint with several reference implementations for sustainable precision agriculture.

The platform has to cover a large number of bases, so the sensor edge is handled using [Red Hat Device Edge](#) with [AWS IoT Greengrass](#) for onboarding. In addition, Red Hat OpenShift AI runs on multi-access edge computing (MEC) with NVIDIA GPUs, supported by IBM watsonx.ai in the cloud.

The project validated and tested three use cases for precision farming around livestock and crop health. For example, the project explored “smart” approaches to two fundamental crop-health processes: weeding and irrigation.

For the smart weeding solution, AI/ML analyzes drone-captured images of farmland to determine which areas require attention (e.g., weed spraying),

with Red Hat OpenShift AI used to identify where the weeds are. The AI generates a map for drone sprayers to accurately apply treatment to affected areas. Having edge-based AI/ML running on MEC allows farmers to make real-time decisions based on information from sensors and other sources, such as weather forecasts, soil conditions, and crop requirements. They can then use this data to create specific drone flight paths for remedial actions. Low-latency real-time processing and 5G edge computing can be applied with feedback to enhance the models and improve efficiency rates.

Smart irrigation begins with an analysis of soil data, such as humidity and temperature, along with weather information like precipitation, wind, and air temperature. At the beginning of the cycle, gauging air temperature allows farmers to infer soil temperature, which guides the best time to plant a particular crop.

The solution also calculates the exact amount of precipitation required to

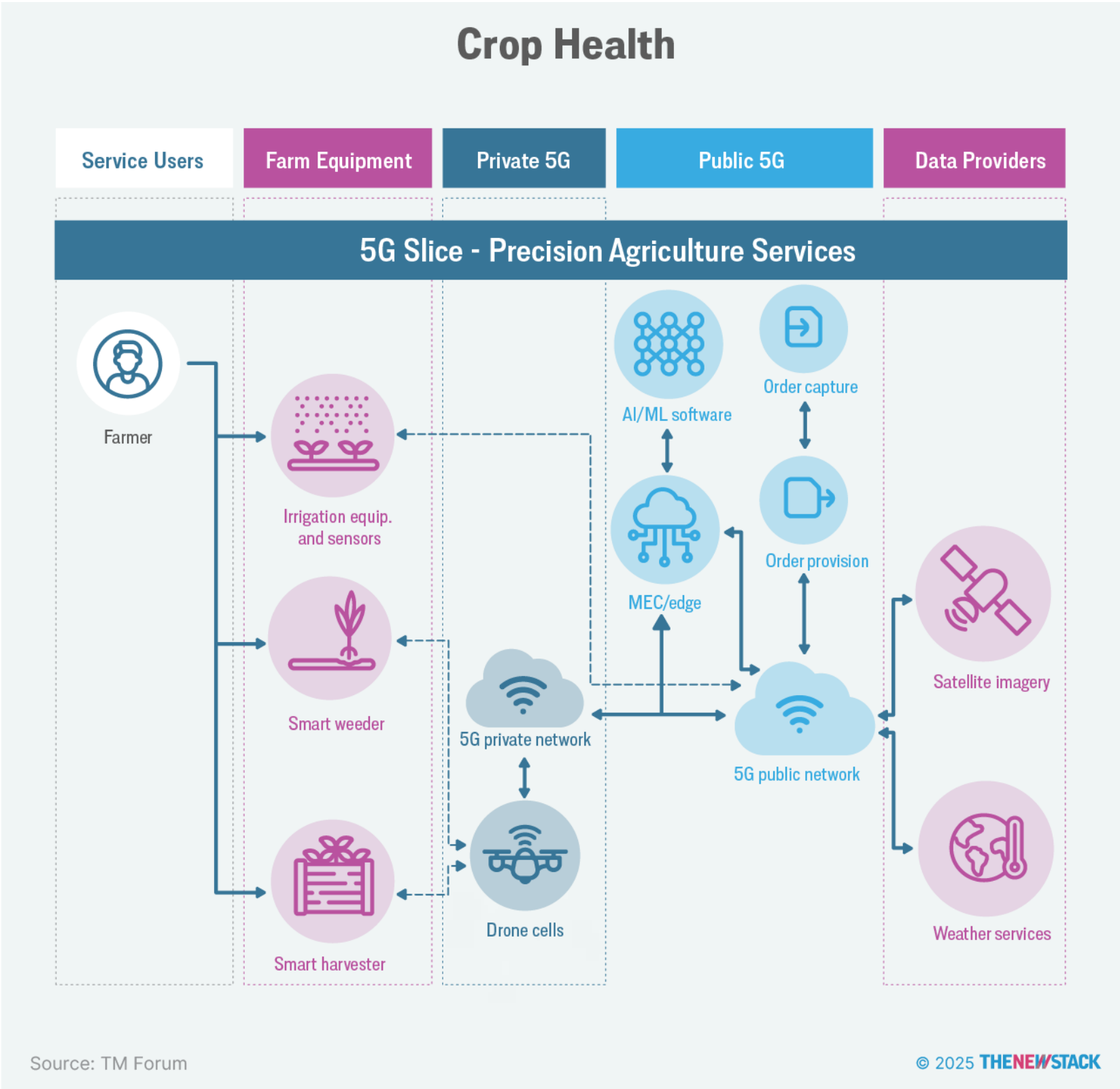


FIG 2.1 The sustainable agriculture reference architecture uses 5G network slicing technology to rapidly deploy software to the edge.

maintain the desired soil humidity profile and nutrient requirements for each crop. This data is used to build a comprehensive farm model. As project team member [David Kypuros](#), global principal solutions architect at Red Hat, explained to me: “The platform creates maps over the soil. Over time, it shows

that every time you plant a legume and an area nearby isn’t responsive to nitrogen, you don’t need to water or drop as much nitrogen there, because it’s not going to help.”

The reference implementations take advantage of 5G network slicing — a

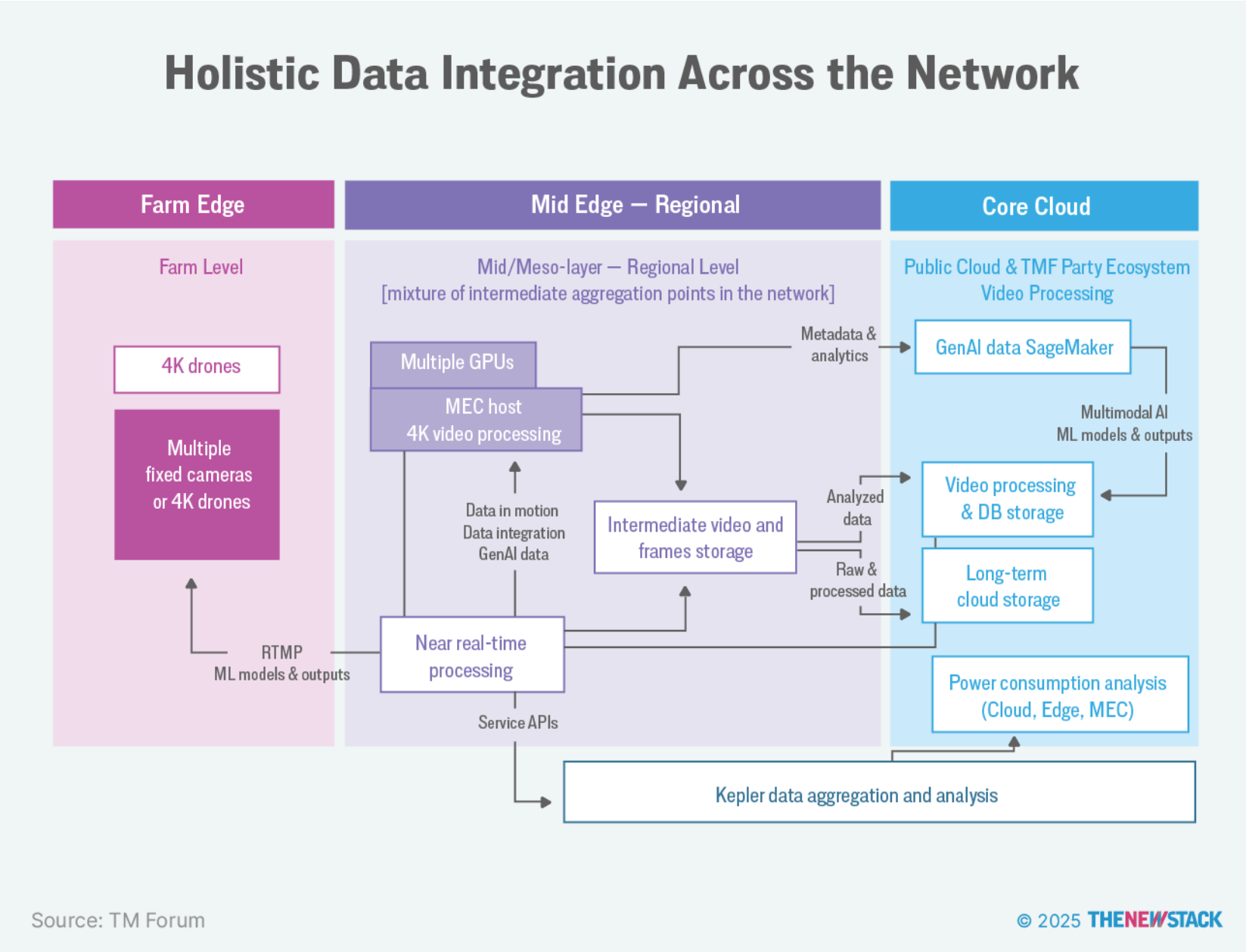


FIG 2.2 The crop health reference architecture uses Kepler to measure energy efficiency and intensity in the cloud native infrastructure.

network architecture that enables the multiplexing of virtualized and independent logical networks on the same physical network infrastructure. This allows a peak throughput of 10Gbps and supports the rapid deployment of software to the edge.

The team also considered environmental impact, using the excellent [Kepler](#) (Kubernetes-based Efficient Power Level Exporter), a Cloud Native Computing Foundation (CNCF)

sandbox project founded by Red Hat’s Emerging Technologies group, to measure energy efficiency and intensity in the cloud native infrastructure (edge and edge computing, workload, and application layers).

Kepler uses [eBPF](#) to probe CPU performance counters and Linux kernel tracepoints. This data is fed into a machine learning model to estimate power consumption from the associated Kubernetes pods.

Energy and Renewables

Power consumption statistics are presented as Prometheus metrics. Telemetry from Kepler can be used for pod scheduling or scaling, energy consumption reporting, and visualization. It can also be extended with carbon intensity metrics to report on a cloud workload's carbon footprint.

The precision weeding solution was tested on an active farm, with a physical survey of the field performed using blue dye to evaluate accuracy. In tests, the project demonstrated 20% to 30% reduction in water and pesticide use.

As many countries have introduced tax incentives to reduce the amount of nitrogen going into the soil, the AI model can be combined with knowledge of local tax laws to reduce nitrogen use. More broadly, the data from the platform can be used by farmers, governments, and others to develop more sustainable agriculture practices.

In oil and gas extraction, “flaring” refers to the release and burning of hydrocarbon gases, the majority of which is methane, a potent greenhouse gas. The World Bank estimates that in 2023, energy companies [flared 144 billion cubic meters of gas](#), emitting the equivalent of over 400 million tons of carbon dioxide. That equals the carbon dioxide released during 9 trillion miles of car driving.

While flaring is sometimes essential for safety reasons, fossil fuel companies often also flare gas as a way of saving money. If they are drawing off only small amounts of gas or drilling in a remote area, they may not think it worthwhile to spend money to collect and transport the gas.

Fossil fuel companies are also keen adopters of edge technology and AI. Aramco, for example, built an AI Hub to analyze the more than 5 billion data points per day generated from wellheads in oil and gas fields. The company says this has enhanced its understanding of petrophysical

properties and expedited decision-making in exploration and drilling.

Its AI model, combined with other techniques, has helped reduce total flaring by more than 50% since 2010, achieving one of the industry's lowest flaring rates.

Aramco is exploring a number of other edge-based AI initiatives to move it toward its stated goal of zero flaring. One example is machine vision — “a product that performs video and image analytics on the flare tip to analyze flare angle, length, and ‘flare-to-smoke’ ratio, predicting flare size before it happens,” [Aramco wrote](#).

It is very impressive tech, and Aramco deserves credit for trying to mitigate some of the climate impacts of fossil fuels.

[PowerFlex](#), a subsidiary of EDF Renewables, is another company exploring edge solutions in the energy and renewables sector. The PowerFlex technology provides renewable energy

solutions designed to help organizations transition to clean energy. The company has the fourth-largest EV charging network and is the second largest installer of commercial solar in the United States.

The PowerFlex X platform supports adaptive intelligent energy management, helping utility companies and building owners provide efficient charging for EV drivers and manage demand at peak charging times.

[Jeff Wright](#), director of hardware product management at PowerFlex, explained how the technology is used for a variety of distributed energy resources, ranging from smart meters to weather stations, solar inverters, batteries, EV charging stations, microgrid controllers, and more.

The PowerFlex system uses a mixture of internal and cloud services to forecast anticipated solar facility and EV load. By combining the forecasts with historical and real-time data, as well as user-defined constraints,

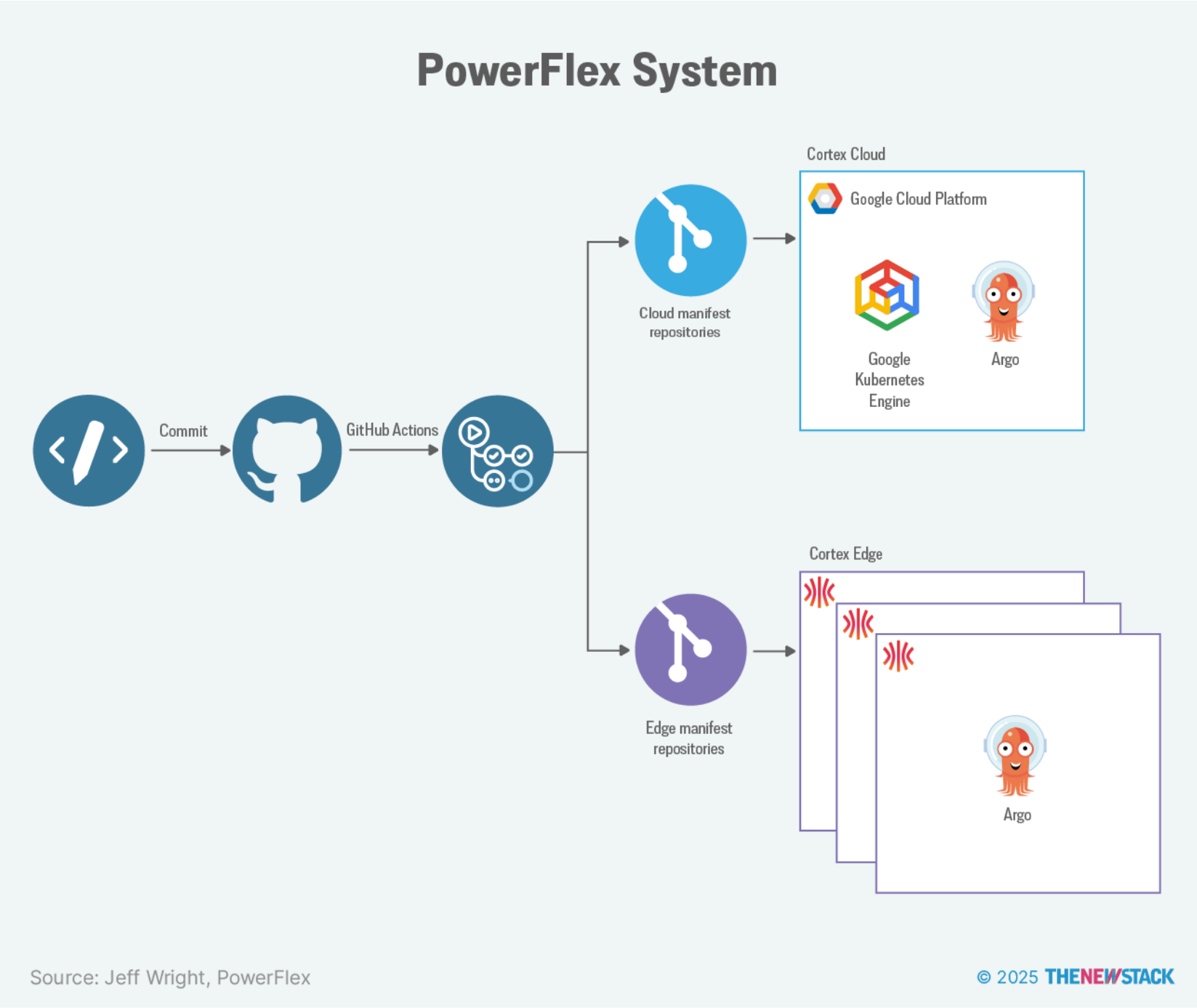


FIG 2.3 PowerFlex uses a fully automated deployment pipeline to deliver its software to its cloud and edge deployments.

the system can autonomously co-optimize and control on-site assets.

PowerFlex has a fully automated deployment pipeline to deliver its software to its cloud and edge deployments. GitHub Actions on certain triggers are used to build the Docker image files and push deployment manifests for cloud and edge computing, with Argo CD, a declarative,

GitOps continuous delivery tool for Kubernetes, used to keep everything in sync.

In PowerFlex’s early startup days, edge deployments were one-offs that manually built a customer-specific ISO image. While this is a reasonable approach at small scale, as the business grew, the team needed a more easily repeatable and automatable solution.

The PowerFlex team opted for Kubernetes at the edge because it had already been using Google Kubernetes Engine (GKE) for cloud-based services.

After trying an alternative, it settled on [Talos Linux](#), a minimal Linux distribution designed for Kubernetes from Sidero Labs, as the edge operating system.

“With Talos Linux, we can build generic images that we can later patch with customer-specific information, allowing us flexible, simple workflows for project commissioning,” explained Wright at TalosCon in 2023. “Talos Linux enables us to tailor our builds to our specific needs, allowing us to provide custom drivers, dynamic IPs and host names, and complex networking rules. And it can be customized to work anywhere.”

PowerFlex also uses Omni, the Kubernetes management platform from Sidero. Omni offers a single management plane for all Talos Linux clusters, allowing teams to provision Kubernetes clusters with less Kubernetes knowledge. It includes built-in authentication with Google and

GitHub accounts, and hosts load-balancing endpoints for each cluster.

“Omni has been a game-changer for us,” Wright said. “It provides seamless deployment workflows, centralized cluster management, and remote support tools for our clusters. We can look at things like cluster health and machine logs, MAC addresses, and other metadata. And we can perform cluster reboots or apply system alterations through patches.” It’s effectively removed the need to send a truck to the edge location to manage the operating system, he said.

These examples from Aramco and PowerFlex illustrate how edge computing and AI are becoming essential tools for addressing energy sector challenges, from reducing harmful emissions to optimizing renewable energy systems. While the technology applications differ significantly, both demonstrate the potential for intelligent edge solutions to create more sustainable and efficient energy operations.

Healthcare

As the energy transition accelerates, companies that successfully implement these distributed computing strategies will likely gain competitive advantages while contributing to broader environmental goals. The key lies not just in adopting cutting-edge technology, but in deploying it strategically to solve real-world problems at the point where energy is produced and consumed.

The healthcare industry is adopting edge computing solutions to process sensitive patient data locally while maintaining strict security and compliance requirements. From pharmaceutical giants managing thousands of distributed lab networks to younger AI companies enabling rapid stroke diagnosis, healthcare organizations are using containerized edge deployments to deliver critical services directly at the point of care. These implementations must balance the need for real-time processing capabilities with the industry's demanding standards for data protection and system reliability.

Founded in 1896, and now employing more than 100,000 staff, Roche is a global leader in pharmaceutical development and health diagnostics. The company is present in thousands of locations around the world.

Historically, each time a team at Roche wanted to deploy a new application, for example a new laboratory diagnostic test, they would provide their own

hardware and software, including cross-cutting concerns such as logging and monitoring. In larger laboratories, with potentially hundreds of applications, this represents a substantial duplication of effort and is hard to manage.

To mitigate this, Roche established a platform team to unify the hardware and common software components with a framework that individual applications can run on top of within an edge cluster.

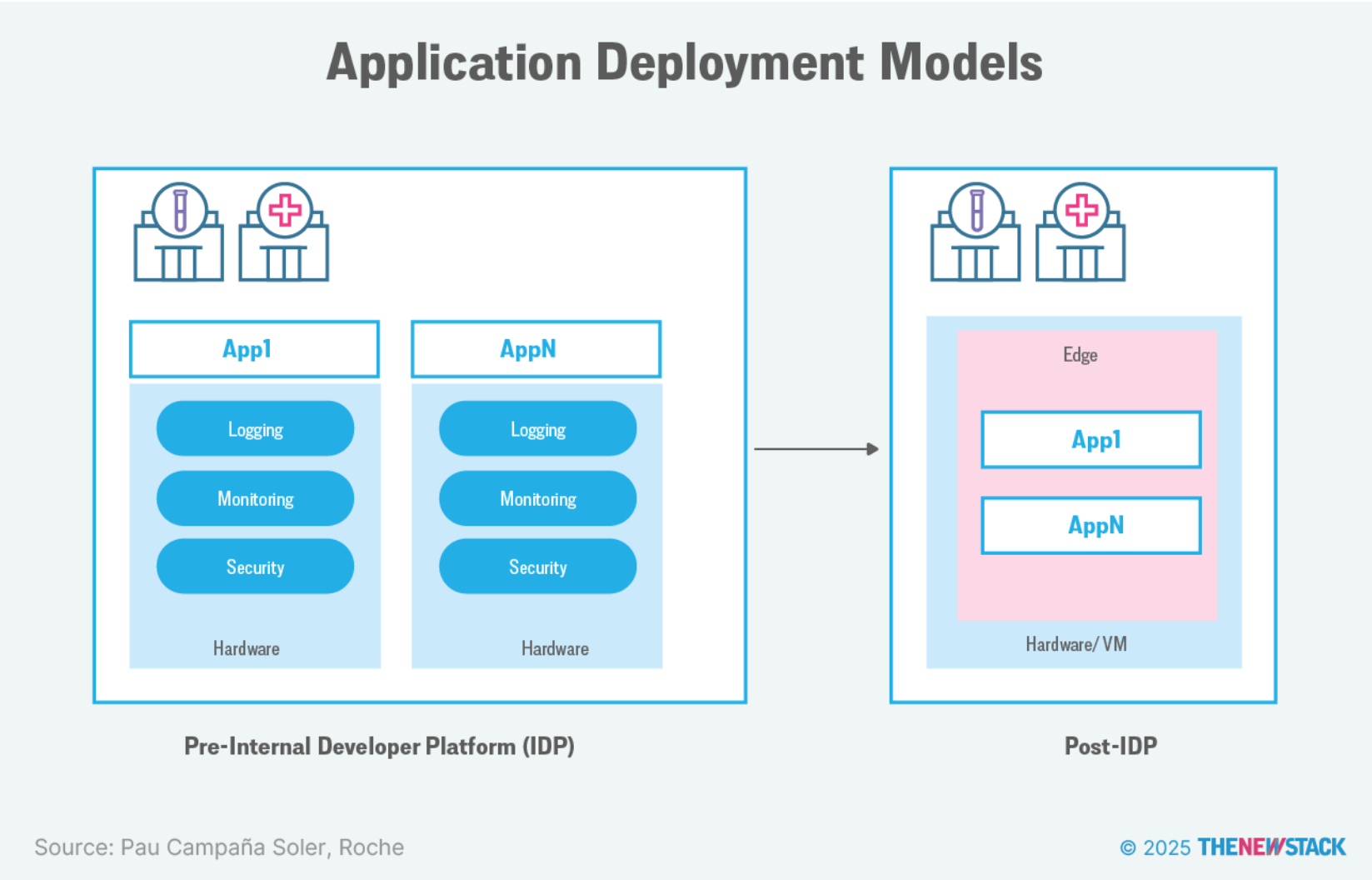


FIG 2.4 To mitigate duplication of resources, Roche unified hardware and common software components to run individual applications within an edge cluster.

A challenge is that the company deploys a large number of Kubernetes clusters to a growing number of labs, hospitals, clinics, and pharmacies; its internal platform team anticipates having around 10,000 edge clusters by 2027. At this scale everything needs to be automated.

The platform comprises two main elements:

- **Fleet management** is handled via a web console that shows the status of all edge clusters and provides centralized access to logs, monitoring, and the registry.

- **Edge clusters** offer an easy way to manage cluster creation, an application-deployment model based on GitOps, and automatic upgrades of the Kubernetes clusters at the edge without the need for technical knowledge about Kubernetes. This approach allows for including standard plumbing components, such as Grafana and Prometheus, by default, leaving developers free to focus on their individual applications.

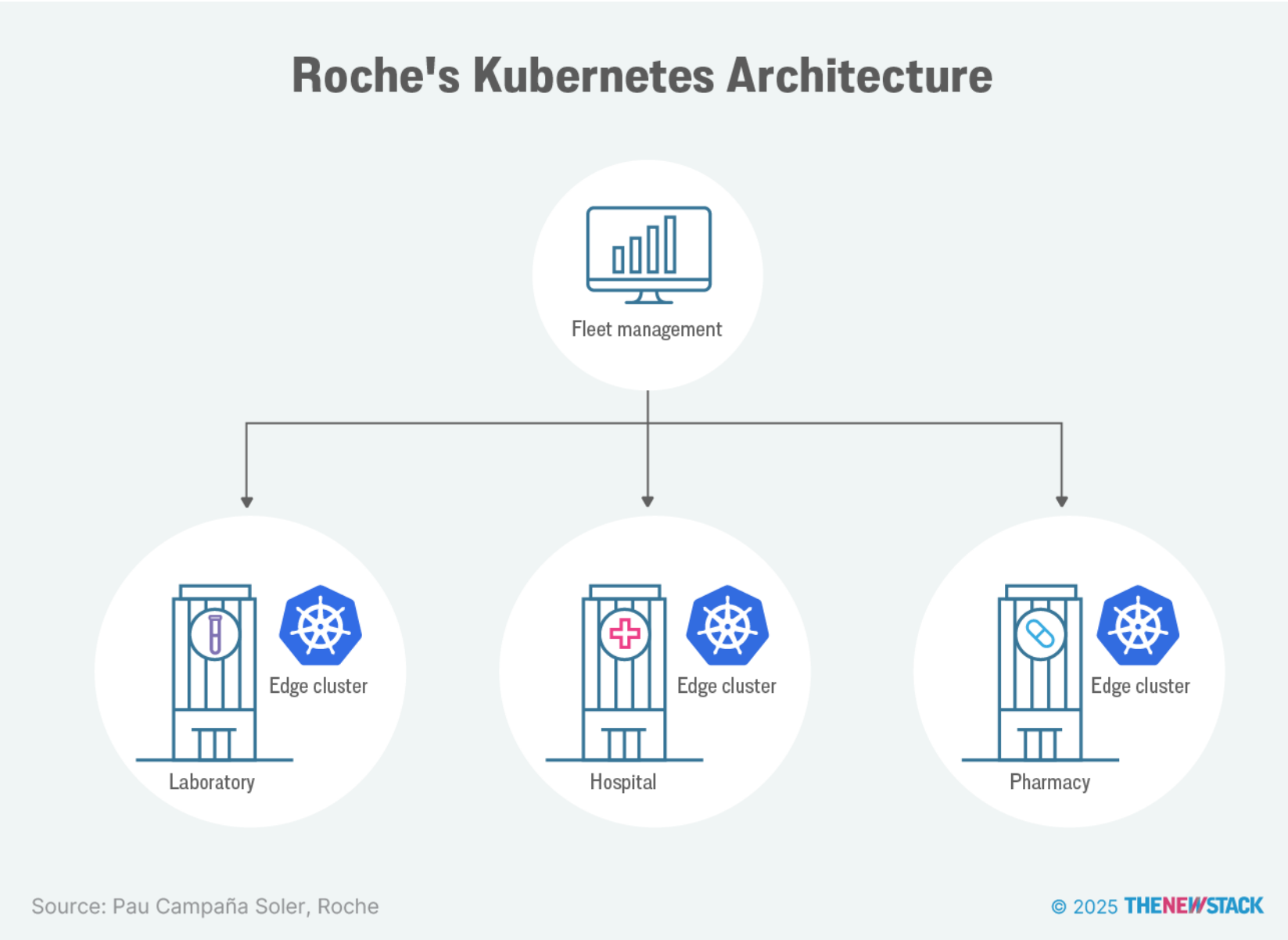


FIG 2.5 Roche manages its fleet using a web console that shows the status of all edge clusters and provides centralized access to logs, monitoring, and the registry.

[Pau Campaña Soler](#), a Kubernetes platform engineer at Roche, explained at TalosCon 2024 how the platform’s first implementation was based on its

own custom Linux distribution, with Rancher Kubernetes Engine (RKE2) as the Kubernetes distribution and Rancher handling fleet management.

The platform team ran this in production with more than 10 clusters but hit issues with complex updates, nonstandard operating system configurations, and security risks. It switched to Talos Linux, citing advantages around immutability and security.

Roche's platform can create two types of clusters:

- **Local-talos:** Used for development, this option creates a Talos Linux cluster using Docker. Local-talos gives developers the same Kubernetes and Talos Linux version they will use in production. "In the past, our users were creating a minikube, or similar kind of cluster, to test their applications before deploying them in a production cluster," Soler said at TalosCon. "Now with Talos Linux, they can test it using the same Kubernetes version and the same operating system version."

- **Baremetal-talos:** For production, an ISO is provided for use on bare metal or in VMs. The ISO takes advantage of Secure Boot, a security standard to make sure a device boots using only software that is verified. When enabled, only binaries that have a trusted signature can be executed.

Because patient data is particularly sensitive, Roche must be extremely cautious with security, and that, along with immutability, were its main reasons for switching to Talos Linux.

"Talos Linux mounts the root filesystem as read-only and removes any host-level access, such as via SSH [Secure Shell], which eliminates configuration drift," Soler said.

Healthcare AI company RapidAI provides another solid example of a [healthcare edge use case](#). RapidAI offers image analysis for magnetic resonance imaging (MRI) and non-contrast and contrast computed

tomography angiography (CTA) scans. Its flagship product has extended the ideal window for treating an ischemic stroke from only six hours to approximately 24 hours.

RapidAI's flagship product is available as a platform and a mobile app, with results distributed via the app and other channels, such as email. The mobile app has built-in workflow and communication capabilities, so clinicians can interact and share images across their team.

The IT team uses a wide range of programming languages.

"All our cloud processing is in Go, but we have used everything from Python to C++, Java, JavaScript, TypeScript, and Swift," [Amit Phadnis](#), chief innovation and technology officer at RapidAI, told me.

The full system, including the AI algorithms, is fully containerized using Docker and Kubernetes, with Spectro Cloud Palette used to deploy and

manage the stack. To prevent hardware-related downtime, it forms a three- or five-node Kubernetes edge cluster, which is tolerant to hardware failure in one or more nodes. Spectro Cloud can support both physical and virtual appliance form factors, although RapidAI just uses the latter.

Within the cluster, any updates happen on a rolling basis, where single nodes are consecutively detached and upgraded before they rejoin the cluster. They maintain an A/B partition, with the previous operating system and Kubernetes version on the A partition. Clusters are monitored using a metrics agent built into the edge cluster, which periodically collects all the system health information.

As healthcare organizations continue to digitize critical patient care processes, these containerized edge computing approaches demonstrate how the industry is successfully balancing the competing demands of real-time processing, data security, and operational scalability at scale.

Retail

Managing thousands of retail locations across diverse geographic regions presents unique technological challenges that extend far beyond traditional enterprise IT deployments. For large retailers, each store represents an edge computing environment with distinct network constraints, security vulnerabilities, and operational requirements that demand both resilience and simplicity.

JYSK is an international home furnishing retailer that opened its first store in Aarhus, Denmark, in 1979. It has more than 3,500 stores and webshops operating in 50 countries.

JYSK's development team writes the majority of its custom software using Java and, increasingly, Go. At a high level, JYSK's IT is a hybrid setup: It runs legacy software and applications with privacy or cost concerns on premises, its online shop apps run in the cloud, and it uses physical edge for the local stores. It doesn't mandate one single approach.

"For us, it just has to make sense for the application's requirements," Ryan Gough, technical product owner at JYSK, told me.

From a technology standpoint, the company is in transition. Taking a leaf out of [Team Topologies](#)' methods, it is using a product-centric approach, building out an [internal developer platform \(IDP\)](#) to unify approaches, including for the far edge.

As part of this effort, JYSK ran an initial proof of concept (PoC) with a lightweight Kubernetes distribution and a similarly lightweight operating system across more than 3,500 in-store systems. Kubernetes was chosen to keep the deployment strategy consistent.

"We use Kubernetes on-prem and in the cloud, so it came down to simplicity; we were trying to ensure that developers and operations can speak the same language," Gough said.

To streamline application management, JYSK adopted GitOps to ensure controlled, versioned application rollouts and minimal configuration drift. Cilium, an open source eBPF-based observability and network tool, provides visibility into, and efficient communication between, applications running in stores.

The PoC was successful, but it revealed a number of issues. Key among them was that when managing thousands of nodes, the size and volume of updates and patches risked overwhelming store networks, potentially disrupting operations. Additionally, the diverse network infrastructure across stores introduced further complications. Centralized traffic routing created bottlenecks, hurting performance. Pulling container images through a central container registry with a pull-through cache proved to be a complex and resource-intensive process.

To address these challenges, JYSK adopted Talos Linux. “What attracted me most was the immutability,” Gough

said. “Stores are at the edge of our topology and therefore considered vulnerable systems. They are susceptible to power failures, environmental impacts such as network speeds and downtimes, and even war-driven scenarios, as with Ukraine. So to have our systems immutable and quickly recoverable is key.”

In addition, the small size of Talos Linux simplified managing traffic routing for the store network infrastructure. “We have to deploy our images to 2,500 places, and we need something so small that we can do that easily. Talos Linux gives us this.”

JYSK also embraced a “cattle, not pets” approach to cluster management — treating individual clusters as replaceable rather than unique — which enables rapid reinitialization and reprovisioning in case of failures. This involved creating custom-built images, then using an automated process to deploy them to in-store systems. HashiCorp Packer automates the creation of Talos Linux images, pulling

the necessary images through the Talos Image Factory API and customizing them with preconfigured settings for consistency across deployments. For provisioning, JYSK uses the cloud-init NoCloud data store to allow local configuration to bootstrap the system.

The deployment then communicates with a provisioning API, enabling a fully automated handoff-node setup.

“If we have hardware failures, we can have our distributor send out a new piece of hardware. On receipt, the store plugs it in, switches it on, and within a couple of minutes they are back to normal. It’s all hands-off, and we can boot up a Talos Linux installation in around 46 seconds,” Gough told me.

When deploying Kubernetes at scale at the edge, Gough emphasized the importance of planning for Day 2 operations — the ongoing management, updates, and maintenance of clusters.

“I underestimated how much work this would be at the beginning,” he said.

Automation plays a crucial role in ensuring scalability and efficiency, particularly with image creation, provisioning, and configuration management.

The challenges JYSK encountered — network constraints, security vulnerabilities, operational simplicity, and Day 2 operations complexity — are universal pain points for any retailer operating at scale. To meet new consumer expectations, retailers must invest not just in edge technology, but in the operational frameworks that make distributed systems manageable and resilient.

The retailers that succeed will be those who, like JYSK, embrace automation, immutability, and “cattle, not pets” philosophies to create technology infrastructures that can scale efficiently while remaining simple enough for store teams to operate reliably in diverse, challenging environments.

CHAPTER 03

The Edge Vendor Landscape: Evaluating Your Options

The case studies in Chapter 2 demonstrate how useful Kubernetes is in edge environments. But they also show something else: Kubernetes is complex, and scale exacerbates that complexity.

Spectro Cloud research indicates that a typical business using Kubernetes now has more than 20 clusters; 6% of companies have more than 100 clusters. Mercedes-Benz was [running close to 1,000 Kubernetes clusters in 2023](#). In some edge scenarios, the numbers are much higher; as we saw earlier, Roche expects to be running 10,000 edge clusters by 2027.

The complexity of Kubernetes at this scale is why Kubernetes management platforms exist. Their primary function is to allow companies to operate this many clusters without hiring huge teams. This is achieved via automation and abstractions, which also include minimizing complexity, as well as securing and monitoring clusters.

Key Decision Factors for Economic Buyers

Edge environments, with their particular constraints around network connectivity and compute resources, are a good fit for Kubernetes management platforms. However, edge-based Kubernetes deployments are still relatively new, which means the vendor landscape is changing rapidly.

“Edge-based Kubernetes deployments are still relatively new, which means the vendor landscape is changing rapidly.

For key economic buyers — those who control the budget and perceive the value and risk associated with a purchase — several factors influence platform purchasing decisions:

- **Connectivity challenges:** In distributed environments, connectivity remains a significant concern, particularly as organizations deploy thousands of clusters across remote locations.
- **Remote management:** Edge locations often lack dedicated on-site IT staff, so remote management from a central location is critical for deploying, monitoring, troubleshooting, and managing edge clusters.
- **Security and observability tooling:** Vendors are increasingly adopting secure supply chain measures and confidential computing paradigms to mitigate risks.
- **Automation:** This is essential for IT teams seeking solutions that

simplify deployment and management at scale.

- **Workload acceleration:** As support for specialized hardware devices like GPUs and tensor processing units (TPUs) becomes a requirement and the efficiency of containerized applications remains essential, workload acceleration is increasingly used for offloading tasks.
- **Operating system flexibility:** If you can use a standard Linux distro, ideally one specifically designed for edge deployments, that is likely a benefit. If not, your platform will need to support multiple operating systems.

These three other commonly overlooked but important qualities should also be factored into any final decision:

- **Developer experience (DevEx):** Look for tools that you and your developers enjoy using.

- **Carbon emissions:** Monitoring your environmental impact should be treated as essential, though in practice this is rarely prioritized.
- **Cost intelligence:** Cost management tooling can provide mechanisms for controlling spend.

Your Kubernetes management tool does not have to meet all of these criteria. If you have an existing toolchain that covers many of the bases, you may want your Kubernetes platform to solve specific problems today and plug in other capabilities as required.

Since essential criteria differ across organizations, no one can assess these tools and make a firm recommendation on your behalf. For example, in some cases, the vendor-provided DevEx may seem overly complicated, but that perception is somewhat subjective and largely depends on your specific environment. As a result, it's rarely discussed in analyst reports.

Market Survey: Leading Kubernetes Edge Vendors

This section provides a brief survey of the market-leading vendors in this sector, based on my independent research.

I conducted this study from May 2025 to July 2025 through a combination of analyst briefings with vendors, along with desk research including talking to customers, installing products, reviewing technical documentation, and looking at source code where available. In cases where the vendor did not participate, I exclusively used desk research.

The following vendors were considered but are not discussed in detail in this chapter for various reasons:

- AWS [EKS Anywhere](#) and [Outposts](#)
- [Canonical MicroK8s](#)
- Mirantis [k0s](#) and [k0smotron](#)
- [Rafay](#)
- [Wind River Studio Cloud Platform](#)

Noninclusion does not mean that the vendor should be disregarded. However, if you are considering

adopting Kubernetes at the edge, all the vendors described below should be in the running as you draw up your shortlist.

Rakuten Cloud Native Platform

Rakuten delivers a Kubernetes solution for edge computing through [Rakuten Cloud](#), a component of a larger suite of telecommunications and cloud services. The Rakuten Cloud Native Platform targets large-scale service providers looking to deploy Open Radio Access Networks (Open RAN), 5G Core, Private 5G, MEC, and enterprise applications.

Rakuten scores well for Day 2 operations. The self-healing features are particularly strong, with declarative policies that enable automated recovery, scaling, and migration of workloads. It supports advanced features such as non-uniform memory access (NUMA) awareness, CPU pinning, HugePages support, affinity/anti-affinity rules, multi-Container Runtime Interface (CRI) (containers,

VMs), and policy-driven orchestration. The platform supports pluggable elements like Container Network Interface (CNI) and Container Storage Interface (CSI), but lacks support for modifying core Kubernetes components like etcd and CRI.

The product offers extensive integration, with Intel CAT, NVIDIA GPU slicing, 5G Core optimizations, and Distributed Management Task Force (DMTF) Redfish orchestration supported, and tight alignment with Google Distributed Cloud (formerly Google Distributed Cloud Edge).

The vendor's zero-touch provisioning framework and cloud native orchestrator offer straightforward rollout across diverse environments. Features like Symworld Site Management and AI/ML-powered workflow automation reduce deployment time.

With regard to networking, the platform has robust support for dual-stack IPv4/IPv6 and integration with secondary networks via tools including Multus.

I could find nothing that suggests Rakuten supports workload acceleration or, indeed, any instruction sets beyond x86.

Rakuten Cloud appears well suited for industries with geographically dispersed infrastructure, such as telecom operators. Its solution is not limited to a single segment, however.

Red Hat Device Edge

[Red Hat Device Edge](#) reached general availability (GA) status in November 2023. It extends the capabilities of Red Hat OpenShift, focusing on lightweight Kubernetes deployments for resource-constrained and remote environments. It integrates Red Hat Enterprise Linux (RHEL) with MicroShift, a minimal, CNCF-certified Kubernetes distribution derived from OpenShift, enabling consistent container orchestration across edge devices.

For RHEL, Red Hat Device Edge adds efficient image-based updates through a git-like approach that transfers only the changed portions of the system. This significantly reduces bandwidth usage and update times compared with traditional package-based updates using OSTree. This image-based approach also provides reliable rollback.

From a connectivity standpoint, Red Hat Device Edge supports both single-stack IPv6 deployments or dual-stack (IPv4 and IPv6) configurations, adding flexibility to infrastructure management. It supports air-gapped deployments by embedding MicroShift into immutable RHEL for edge images, enabling offline installation and application deployment. Note, however, that Linux support is limited to RHEL; other distros are not supported.

Workload acceleration is a strong feature of Red Hat Device Edge. It covers a wide variety of accelerators such as GPUs, data processing units (DPUs), and field-programmable gate arrays (FPGAs), making it a strong contender for high-performance

computing needs in resource-constrained or disconnected scenarios. With a focus on industrial applications, Red Hat recently announced new low-latency and near-real-time capabilities, including latency below 1ms and deterministic performance for consistent outcomes. The vendor is also building out AI capabilities and has announced NVIDIA Jetson Orin and NVIDIA IGX Orin system support in tech preview.

Deployment automation can optionally be handled via the Red Hat Ansible Automation Platform, with support for Helm charts and operators. Ansible Automation Platform supports provisioning applications, configuring clusters, deploying applications, setting up security, meeting compliance requirements, and avoiding configuration drift.

Ansible Automation Platform is geared more toward operators than developers and has a small learning curve with YAML-based, human-readable playbooks. It is declarative, meaning that you describe the desired state, and

it's idempotent in the sense that it will make only the required changes and is repeatable without side effects.

The picture in terms of fleet management scenarios is more complex, however. Red Hat Device Edge also integrates with Red Hat Advanced Cluster Management for Kubernetes (as well as Ansible Automation Platform). Advanced Cluster Management enables consistent security policies and configurations across all edge locations. This approach provides flexibility, but it lacks consistency since multiple tools — console.redhat.com, Advanced Cluster Management, and Ansible Automation Platform — are required to support different edge deployment models and levels of connectivity.

OpenShift supports some customization of pluggable elements like CNI and CSI, but it restricts modifications to core Kubernetes components to maintain stability.

Likewise, while Red Hat Device Edge would seem a logical choice for existing

OpenShift users, it is worth noting that Red Hat OpenShift and Red Hat Device Edge have distinct operating models and approaches, which can be frustrating.

Red Hat Device Edge is suited to a wide variety of use cases. Its low latency and growing AI capabilities mean it is particularly suitable for certain applications, such as industrial Internet of Things (IIoT), autonomous vehicles, online gaming, and smart cities.

Scale Computing HyperCore

Founded in 2008, [Scale Computing](#) shipped its first hyperconverged product, HyperCore, in 2012.

The Scale Computing Platform comprises three elements:

- **Hardware:** Scale Computing-branded appliances, such as the HE100 Series, are suitable for edge environments. Under the

hood, the HE100 machines are based on Intel NUCs, while the 500 series is largely based around Lenovo equipment.

- **HyperCore:** This Linux-based operating system runs on top of the nodes and includes disaster recovery capabilities. HyperCore includes an integrated Kernel-based Virtual Machine (KVM)-based hypervisor; a purpose-built software-defined storage layer, Scale Computing Reliable Independent Block Engine (SCRIBE); and the Autonomous Infrastructure Management Engine (AIME), which orchestrates all HyperCore cluster activities.
- **Fleet Manager:** This web-based Software as a Service (SaaS) offering provides monitoring across the fleet of HyperCore clusters.

HyperCore aims to support a wide variety of setups. These include legacy VMs, container runtimes (both individual

Docker containers and Kubernetes clusters), and as wide a variety of edge container management strategies as possible, including Rancher, OpenShift, Avassa, and Google Anthos. Previously, HyperCore was available only through Scale Computing's hardware, but the vendor now sells software-only subscriptions.

Taking the same approach as the public cloud providers, HyperCore runs containers inside virtual machines. Users can deploy container-optimized operating systems, such as Talos Linux with containerd, or other container runtimes such as Docker or CRI-O with other operating systems, within these virtual machines. This allows for the deployment and management of Kubernetes clusters alongside other applications, including legacy applications, on the same platform.

This VM-based approach works particularly well for smaller Kubernetes deployments, such as the single-node Kubernetes clusters commonly found in edge environments. Any Kubernetes cluster running in a VM on top of a

Scale Computing three-node cluster gains storage resilience, for example. HyperCore also has its own lightweight container orchestration engine, simplifying container deployment, scaling, and recovery.

HyperCore automation has traditionally been provided via a REST API that enables Infrastructure as Code (IaC) for a single cluster. The vendor also has a newer REST API for Fleet Manager.

Alongside these APIs is a Hypercore Ansible Collection, which provides automation via Ansible and offers support for provisioning applications across potentially thousands of HyperCore systems. Ansible's advantages are the same for HyperCore as they are for Red Hat Device Edge: small learning curve; simple, human-readable YAML playbooks; declarative configuration; idempotency. It also offers support for HashiCorp Terraform, a Business Source Licensed (BSL) IaC tool that enables automated infrastructure provisioning and management across cloud and on-premises environments.

Most of the edge-to-cloud communication within Scale Computing's platform is MQTT-based. For security, Scale Computing has Secure Link connectivity for Fleet Manager. This is a secure reverse tunnel providing capabilities similar to a virtual private network (VPN). In addition, the vendor offers encrypted data transmission via Transport Layer Security (TLS), Secure Boot mechanisms, access control policies, and role-based access control (RBAC) administration.

Scale Computing also offers one-touch provisioning (OTP) via USB for air-gapped or dark site installations, enabling secure provisioning without cloud connectivity. HyperCore has a separate user interface (UI) from Fleet Manager. While convenient for air-gapped or dark site installations, it has a different look and feel from Fleet Manager, and users will need to switch between different interfaces depending on what they are trying to do.

GPU-based workload acceleration is offered for select NVIDIA GPUs in

certain Scale hardware appliances or third-party systems. There is currently no support for DPU or FPGA.

Scale Computing has a service practice to assist from pilot stage to mass deployment. It also offers a growing suite of “blueprints” for best-practice deployments of common solutions.

Although traditionally focused on small and midsized businesses, with Fleet Manager, HyperCore is expanding to cover larger edge estates in industries from retail to manufacturing and healthcare. The offering is strongest for hardware and application deployment. Conversely, for container runtimes, Scale Computing offers a great deal of flexibility but requires customers to make a lot of decisions and potentially set up their environment to deploy containerized workloads to the edge.

Sidero Talos Linux and Omni

[Sidero](#) focuses on facilitating Kubernetes deployments at the edge through its Omni platform for Kubernetes cluster management and tight integration with [Talos Linux](#), a minimal and immutable Linux distro. Omni emphasizes security, consistency, and easier maintenance in distributed environments.

Sidero’s offering is the most opinionated of the platforms I researched. Many of its competitors go further up the stack into applications with built-in options for CI/CD, such as Argo, Flux, or GitOps approaches. Or they go more horizontally across providers, using the cluster API to manage Amazon Elastic Kubernetes Service (EKS), Google Kubernetes Engine (GKE), and on-premises clusters, as well as edge using a single, if not unified, interface.

Sidero has chosen to stop at the Kubernetes layer. For CI/CD pipeline

According to [Andrew Rynhard](#), CTO of Sidero Labs, Talos Linux emerged from real-world pain points experienced by infrastructure managers who were burning out — overwhelmed by repetitive tasks, complex automation layers, and bloated systems.

Rather than adding more automation tools, Rynhard chose a radical approach: Eliminate complexity at its source. The core philosophy is to “Run and be responsible for less software,” [Rynhard wrote](#), “because less software doesn’t mean less power. It means less stress, fewer CVEs, and more time to be creative, to build, or to simply get out and live.”

Talos Linux reimagines the operating system for Kubernetes. While systems like Ubuntu contain more than 2,000 binaries, Talos Linux operates with fewer than 50. It strips away nonessential functions like time-zone settings, user management, and package management, eliminating tasks that traditionally required ongoing oversight and troubleshooting.

By removing layers instead of adding them, it reduces both the cognitive load and potential failure points, allowing teams to focus on meaningful work rather than infrastructure maintenance.

That operating system now forms the cornerstone of Sidero.

management or anything further up the stack, you will need other tools.

When considering its offering, Sidero’s reliance on Talos Linux is important to weigh. Talos Linux is a declarative, immutable, and extremely minimal Linux distribution that was specifically redesigned for running Kubernetes. It also is particularly well suited to edge deployments.

To achieve this, Sidero made some radical decisions; the most notable is that there is no Bash, SSH, or systemd. Instead, Talos Linux is API-managed and configuration-driven. The API is also how you get debugging data such as networking information, a list of containers, pods, or interfaces, and details on how they are configured. Communication is bidirectional via mutual TLS (mTLS) encrypted traffic using Google Remote Procedure Calls (gRPC).

To change anything, you submit a configuration file via the API. That configuration file is a declarative YAML file that looks and feels a lot like a

Kubernetes object. Rather than systemd, Talos Linux comes with its own PID 1, written entirely in Go. PID 1, which is the first Linux user-mode process created, runs until the system shuts down and manages the services.

When you submit a new configuration file, controllers and reconcile loops that live within PID 1 try to reconcile the state that you want into the state that's running on that machine. This is all handled in the background using Linux APIs and by manipulating files. Talos Linux also provides Kubernetes-specific guardrails. Once in use, the operating system fades into the background; in effect, Kubernetes becomes the operating system.

This approach fits well for operators familiar with Kubernetes but carries a steep learning curve for those expecting a more conventional Linux experience. Organizations looking to adopt Talos Linux would likely need to factor retraining operators into their decision-making. Conversely, for organizations with less Kubernetes experience, Talos Linux simplifies Kubernetes deployments.

These design choices also impose certain restrictions; for example, an organization would need to verify that its preferred storage solution and security scanner can work in the absence of a shell.

Sidero scores well for security. The small size of Talos Linux substantially reduces the attack surface and maintenance overhead. Sidero offers Trusted Boot (a combination of disk encryption and Secure Boot) to ensure you are running the version of Talos Linux you think you are. Both the Linux distribution and Kubernetes are hardened, the former consistent with the Center for Internet Security (CIS) Benchmark.

A software bill of materials (SBOM) is a critical component for tracking what software is in use and mitigating Common Vulnerabilities and Exposures (CVE). Sidero publishes SBOMs for Talos Linux, which can be integrated into traditional security tools, and it provides SBOM information directly from the Talos Linux machine API.

Federal Information Processing Standards (FIPS) compliance is

important for regulated and highly secure environments. It defines which cryptography can be used on systems and for communication between secure endpoints. Sidero has FIPS 140-3-compliant options for Talos Linux, Omni, and all command-line tools. Complete FIPS-compliant Kubernetes components will be available by the end of 2025.

The offering also scores highly with regard to connectivity, with dual-stack IPv4/IPv6 networking and Multus CNI support.

Remote management is another strength. Sidero's technology can scale across thousands of clusters, with centralized and API-driven control. WireGuard-based remote access ensures secure oversight in restricted firewalled locations. When Talos Linux first boots, it configures the Linux kernel to set up a point-to-point WireGuard tunnel with Omni. WireGuard only needs one end of the tunnel to be public — you can manage the machine if it has outbound egress, and it doesn't need public internet protocols (IPs) to

do it. As a result, administrators can unify multiple deployments under a single management console, saving time and effort.

For air-gapped environments, Talos Linux 1.9 adds [image cache](#) for preseeding container images as part of the Talos Linux installation media. This can eliminate the requirement to run additional registries or maintain hosts outside of the Kubernetes cluster. The resulting Talos Linux installation ISO can be copied to a USB stick for installation on-site.

Regarding workload acceleration, Sidero supports multiple GPU drivers, DPUs, single-root I/O virtualization (SR-IOV), and Data Plane Development Kit (DPDK) network accelerators. It lacks official FPGA support.

Omni allows customizing CNI, CRI, and some CSI providers, but technical interchangeability for core components like etcd is not officially supported.

Sidero is the “easy button” for Kubernetes. Its vertical integration

approach helps newer users get Kubernetes up and running, with near-seamless edge cluster updates. I like this approach, and believe that its flexibility, as it relates to edge Kubernetes, derives from its intentionally opinionated design, with minimal dependency on a thin underlying operating system that serves as the interface between the hardware and the Kubernetes interface.

Talos Linux and Omni are a particularly good fit for the far edge. If you imagine a retail store with a couple of machines to handle point-of-sale and anti-theft functions, Sidero's offering makes it very easy to manage this kind of setup.

Talos Linux is often compared to small footprint Kubernetes installers such as Kubeadm or K3s, and the CNCF sees it this way. However, that isn't really correct; it's a minimal Linux rather than a minimal Kubernetes.

In my testing, Talos Linux was the most lightweight option for production-ready multinode Kubernetes when

considering CPU and memory utilization, disk and network I/O, and disk usage, a finding [confirmed](#) by Justin Garrison, head of product at Sidero. This makes it the best choice among the vendors I reviewed for highly hardware-constrained environments. Indeed, the small size and easy administration of Talos Linux makes it a good option for a single machine at the device edge. There are other edge Linux distros, but Talos Linux includes everything you are likely to need, and nothing else.

At a larger scale, for midsize and large enterprises that can accept a tailored, integrated stack rather than a "best-of-breed" approach, Omni's centralized management and immutable design can reduce operational overhead and risk, especially in widely distributed edge scenarios. As such, Omni is ideal where robust security and consistent operation in distributed settings are top priorities. It targets general-purpose edge environments across industries like retail, manufacturing, and telco that need streamlined Kubernetes

administration and low on-site maintenance.

Along with JYSK and PowerFlex, reviewed in the case studies chapter, other companies using Talos and/or Omni at the edge include Berkshire Grey, CrossnoKaye, Forterro, Hathora, Nokia, and SNCF.

Spectro Cloud Palette

[Spectro Cloud](#) specializes in Kubernetes life cycle management. Its flagship solution, Palette Edge, is part of the broader Spectro Cloud Palette suite. It includes a CNCF-compliant edge-optimized distribution of Kubernetes called Palette eXtended Kubernetes-Edge (PXK-E), which provides centralized management for Kubernetes clusters.

Palette provisions workload clusters on edge hosts from the Palette management console. Palette also provides end-to-end cluster management through scaling,

upgrades, and reconfiguration operations.

Spectro Cloud Palette has made considerable improvements to its security capabilities since I [assessed it](#) a year ago, and Palette Edge now scores highly here. In partnership with Intel, the vendor created Secure Edge-Native Architecture (SENA), a security framework designed specifically for the edge. Palette includes a range of security controls, such as SBOM vulnerability scans, immutable operating system images, hardened configurations, and Trusted Boot.

For governments, regulated industries, and other security-focused applications, PXK-E can add FIPS cryptography to the distro and the Kubernetes clusters it deploys; this is branded as Palette VerteX, which can be combined with FIPS (or indeed non-FIPS) Kubernetes distributions including RKE2 and K3s. Note that because of this strong approach to security, Palette Edge provides limited support for customizing core

Kubernetes components like etcd or the CRI.

Automation is also well supported. Spectro Cloud's range of low-touch and zero-touch deployment options streamlines deployment across distributed edge environments. The zero-touch deployment tooling supports air-gapped scenarios, and Palette's Local UI means that independent software vendors and other field technicians get a simple, consistent experience when working on disconnected clusters. This reduces risks of human error in potentially difficult or dangerous situations in the field.

Regarding Day 2 operations, an agent on the clusters performs local policy enforcement to maintain the desired state. Palette can also automatically renew certificates for local clusters, handle upgrades and patches for anything defined in the cluster profile, and deliver a baked-in backup/restore feature through open source data migration and protection tool Velero.

For networking, Spectro Cloud offers support for dual-stack IPv4/IPv6. Since several of the control plane components in a Kubernetes cluster require stable IP addresses, Spectro Cloud also has a tech preview for a feature that allows you to create a virtual overlay network on top of the physical network, where the virtual IP addresses of all cluster components are managed by Palette. Inside the cluster, the different components use the virtual IP addresses to communicate, rather than underlying IP addresses that can change due to external factors.

Spectro Cloud offers high availability by forming a two-, three-, or five-node Kubernetes edge cluster, which is tolerant to hardware failure in one or more nodes. Within the cluster, updates happen on a rolling basis, where single nodes are consecutively detached and upgraded before they rejoin the cluster. It also uses an A/B partition architecture, with the previous operating system and Kubernetes versions on the A partition.

To monitor clusters, Spectro Cloud has a metrics agent built into the edge cluster, which periodically collects all the system health information.

Built-in support for KubeVirt at the edge, which the vendor calls Palette Virtual Machine Orchestrator (VMO), means that if you receive software from a vendor as a VM image, you can run it on the same edge box as your other software.

Spectro Cloud's approach to the operating system is based on its customers building a customized, immutable operating system from a variety of Linux distributions. The default is openSUSE, but customers can also install Ubuntu, RHEL, or another custom operating system on the edge host, using Spectro's EdgeForge to customize the distribution. This is customization at the point of creation rather than individualized customization on the box. EdgeForge outputs an ISO or Kairos-based image that allows customers to build a Palette Edge Kubernetes cluster

that can be deployed across the fleet. A deployment mode called "agent mode" allows companies (such as telcos) with a preferred method of building a cluster to provide the infrastructure and operating system.

Workload acceleration is slightly limited when compared to some competitors, although RapidAI, Nokia, Tevel, Dentsply Sirona, and GE HealthCare all run AI workloads via Palette. Spectro Cloud supports both DPUs and GPUs. GPUs are supported via a verified NVIDIA operator pack. (In Palette, Packs are the profile layers representing a cluster's desired state.) NVIDIA BlueField-3 DPU support is offered through a validated architecture. There is currently no official support for FPGA.

Palette Edge is designed as a complete solution rather than a feature-specific tool, suitable for greenfield deployments or replacing legacy systems, with a broad focus that makes it ideal for general-purpose edge computing scenarios. It is highly scalable. [Romain Decker](#), principal

product manager at Spectro Cloud, described how a retailer [scaled Palette to 10,000 Kubernetes clusters](#). It is highly flexible with regard to the host operating system, and it can support both VMs and containers, plus a wide range of hardware and applications. This approach, however, adds some risk and setup complexity when compared to more opinionated options.

SUSE Edge Suite

[SUSE Edge Suite](#) is a Kubernetes-based solution that consolidates multiple SUSE components — such as Rancher Prime for multicluster management, K3s and RKE2 for lightweight cluster orchestration, and edge-specific tooling — to streamline deployment and operations in edge environments.

The product has advanced remote management capabilities. It combines centralized multicluster management via Rancher Prime with secure “phone-home” provisioning through Elemental. This enables late binding of connected

edge nodes to desired Kubernetes clusters and application deployment, plus GitOps-based updates via Fleet. These features enable seamless operation across connected and disconnected environments, supporting up to tens of thousands of clusters. The platform’s ability to efficiently manage bandwidth-constrained or intermittent-connectivity scenarios makes it ideal for scaling operations in geographically dispersed edge deployments.

Optional full bare-metal life cycle and management support with Metal3, MetalLB, and Cluster API (CAPI) infrastructure providers enables full end-to-end provisioning of bare-metal systems that have remote management capabilities.

For air-gapped environments, SUSE Edge Image Builder allows customers to generate fully customized installation media that contains all of the required deployment artifacts to enable single-node and multinode, highly available Kubernetes clusters at the edge. These include any workloads or additional

layered components required, without any network connectivity to the outside world, and without the intervention of a centralized management platform.

Its security approach is also strong. SUSE Security provides image vulnerability scanning, deep packet inspection, and real-time threat and vulnerability protection. RKE2 is a hardened and certified Kubernetes distribution intended for use in government and regulated industries.

Note, however, that the platform doesn't support emerging features such as remote attestation. This may limit its appeal in some highly regulated industries or other scenarios requiring the highest levels of supply chain verification and hardware trust.

SUSE Edge Suite has impressive customization capabilities. It allows modification of pluggable elements like CNI, CSI, and CRI, and core Kubernetes components such as etcd.

SUSE Edge Suite has limited operating system support, focusing solely on

SUSE Linux Micro, an immutable Linux optimized for containers, self-healing, and edge workloads. While this vertical integration promotes stability and performance, it restricts organizations that require support for third-party operating systems or broader compatibility with non-Linux platforms like Windows or macOS.

SUSE Linux Micro supports x86 and ARM CPUs. Workload acceleration is available for NVIDIA GPUs via prebuilt open source drivers on SUSE Linux Micro, with support provided by both SUSE and NVIDIA in collaboration.

SUSE Edge Suite supports diverse industry segments, including retail, manufacturing, and telecom. It is particularly well suited for large-scale edge deployments.

How To Choose the Right Edge Kubernetes Platform

The rise of Kubernetes combined with Broadcom's acquisition of VMware has produced a great deal of churn in the broader infrastructure market, with organizations open to new ideas and approaches.

Given this, it is unsurprising that the edge Kubernetes market is in transition, and traditional "one-size-fits-all" approaches are giving way to specialized solutions tailored for distributed computing challenges. While established players like Red Hat and SUSE leverage their enterprise Linux heritage to deliver comprehensive platforms, newer entrants like Sidero demonstrate how opinionated, purpose-built approaches can dramatically simplify complex edge deployments.

Your choice of vendor ultimately depends on your organization's existing infrastructure investments, specific edge requirements, and tolerance for complexity versus control. As the market continues to mature, you can expect consolidation around platforms that can effectively balance the competing demands of simplicity,

security, and scalability. You can also expect to see vendors continue to add workload acceleration and AI-related features.

I predict the most successful vendors will be those that can abstract away Kubernetes complexity while preserving the flexibility that makes it valuable for edge computing in the first place.

“The edge Kubernetes market is in transition, and traditional ‘one size fits all’ approaches are giving way to specialized solutions tailored for distributed computing challenges.

CHAPTER 04

Kubernetes at the Edge: Applying Tips and Tricks

During my 30-odd years working in software, I've seen five major shifts in how enterprise software is built and deployed: the adoption of higher-level languages like Java and C#; the agile movement; public cloud; DevOps and particularly continuous delivery; and microservices. Each ultimately succeeded because it reduced time to value. It enabled someone in a business to go from having an idea to getting it in front of a customer faster and at lower cost. This combination allows businesses to [try more ideas](#) and use the market's response to determine which ones resonate and which don't. For all the hype and excitement, it remains to be seen whether generative AI will be a sixth.

In many ways, the rise of containerization at the edge mirrors this historical pattern. Reusing practices that the cloud enables, such as CI/CD, supports a more developer-centric framework, even in distributed physical environments. By enabling applications to be packaged in universal formats that operate identically across development, testing, and production environments, containerization simplifies otherwise complex deployment processes.

As we continue to explore the role of AI in the enterprise, the edge represents the next frontier in this evolution. Edge devices are getting more capable. Real-time inference, data locality requirements, and latency constraints all push compute closer to where data is generated and decisions are made.

“The idea is to slowly increase investment in learning and information gathering until you uncover the best approach for your organization.

The Edge of Disaster

The edge isn't simply “cloud computing, but closer.” Rather, it extends cloud native practices to locations where traditional cloud assumptions break down. The edge is a uniquely challenging environment that requires thoughtful attention to these factors:

- **Limited or no IT staff:** If something goes wrong with the infrastructure in an edge environment, you may not have anyone locally with the expertise to deal with the issues.
- **Physical constraints:** Devices deployed to the edge are limited in CPU/GPU power and memory. When compared to a Tier 1 data center, power is often unreliable, and the environment might be unusually hot, cold, or dirty, which can increase fail rates.
- **Security:** In a data center, physical access can be easily controlled and limited; that simply isn't true in an edge environment. This means you

How To Assemble a Team

must pay careful attention to who has access to the devices on the network's edge.

- **Connectivity issues and outages:** Connectivity may be limited, so if you have to rely on the internet, you may not be able to run your applications.
- **Unpredictable demands:** In more mature enterprises, you are likely to find legacy applications that have been running on premises for decades now running alongside modern cloud native containerized applications.

“The organizations that master cloud native practices at the edge will be best positioned to harness real-time insights, respond to local conditions, and deliver capabilities and experiences that were previously impossible.

To apply cloud native practices to the edge, you need a team; that may mean assembling a team from existing employees or hiring new team members. Either way, since edge deployments make use of everything from the physical properties of semiconductor electronics to engineering high-level architectures that span devices and the cloud, you will need a team with broad, cross-disciplinary knowledge, direct experience working on the problem domain, and comfort using an iterative development process.

The minimum capabilities you need are:

- A domain expert with deep insight into the problem you're trying to solve.
- An embedded-device engineer with experience developing for devices similar to the target.
- A cloud native engineer with experience using modern DevOps practices, containerization, and Kubernetes.

Gradually Raise the Stakes

When building your edge team, look beyond technical skills. This is still new territory, which means you should seek individuals who are curious, adaptable, and collaborative.

In addition to domain expertise, the other essential attribute you should aim for is diversity. As Daniel Situnayake and Jenny Plunkett discuss in [“AI at the Edge,”](#) one of the best defenses against societal harm is to build a team with a range of perspectives. This only works if you also have a culture of psychological safety. Even in the best working environments, it can feel risky for employees to speak up if they think their feedback may disadvantage them.

Also remember that [programming is like writing](#). It is a creative process that requires a degree of freedom and latitude. Expect a certain number of false starts; experimentation is necessary to uncover the best ideas. As with the cloud, by fostering an environment where your tech talent can explore and innovate, you’re more likely to unlock the true potential of your edge deployments.

Once your team is in place, the next task is building up their expertise. For example, JYSK started its Kubernetes edge deployment journey using applications that were not business-critical. I also strongly recommend running Kubernetes alongside existing hypervisors, such as VMware and Proxmox, if you already have edge deployments. Regardless, it’s best to start with noncritical workloads before moving to mission-critical applications.

The idea is to slowly increase investment in learning and information gathering until you uncover the best approach for your organization. This is analogous to “gradually raising the stakes,” as Pini Reznik, Jamie Dobson, and Michelle Gienow explain in [“Cloud Native Transformation.”](#)

In [“Visionaries, Rebels, and Machines,”](#) Jamie Dobson described how the size of the risk is determined by how clear the future is.

“If it’s clear enough, then companies can prepare for it with simple ‘no regret moves,’” Dobson wrote. “If the future is

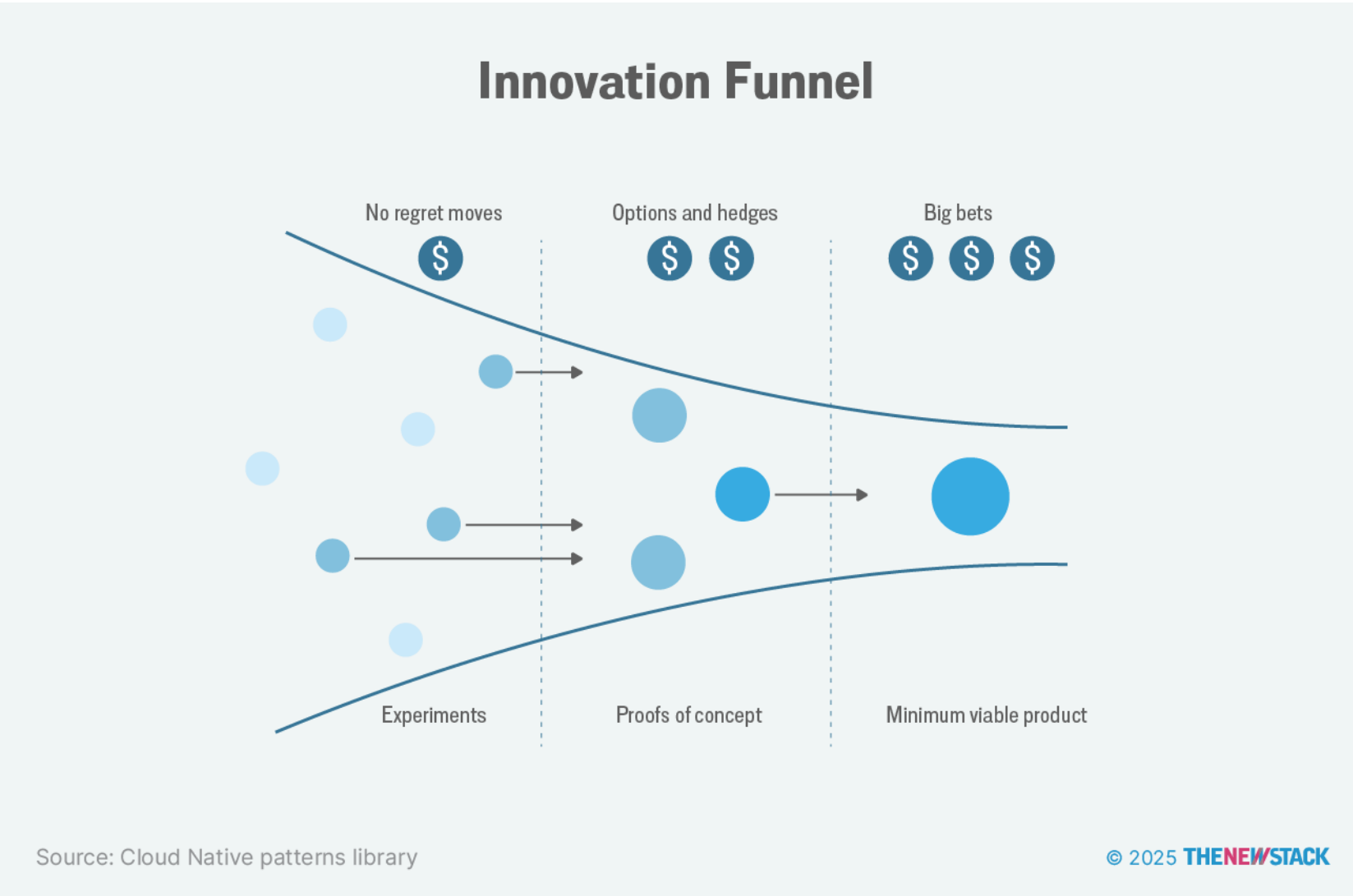


FIG 4.1 Minimize risk by starting with experiments that are not mission critical, and slowly increase your investments as you accumulate expertise.

a bit less certain and there are a few alternatives, the company’s job is to lower uncertainty through experimentation.”

Edge computing represents a significant shift in operational patterns, and organizations need time to adapt their processes, skills, and expectations. This graduated approach offers several advantages. Starting with noncritical workloads allows teams to learn the operational patterns of edge Kubernetes, identify potential issues in a low-risk environment, build

troubleshooting skills and runbooks, and validate monitoring and alerting systems.

Running new systems alongside existing ones provides fallback options if the new system fails. It provides comparison points to validate performance and reliability, as well as gradual migration paths for existing workloads. It also helps build stakeholder confidence through demonstrated reliability.

Prioritize Day 2 Operations

There are two common infrastructure deployment approaches — and both are suboptimal.

Physical: This is most commonly used for more limited edge deployments. Bespoke resources are assembled for each application that will be deployed to the edge sites, and those resources

are sent by truck to every individual site. This creates friction every time you want to deploy a new application, and you amass a disparate collection of systems that you have to manage and monitor separately.

Virtualization-based: Virtualization typically involves two virtualized hosts

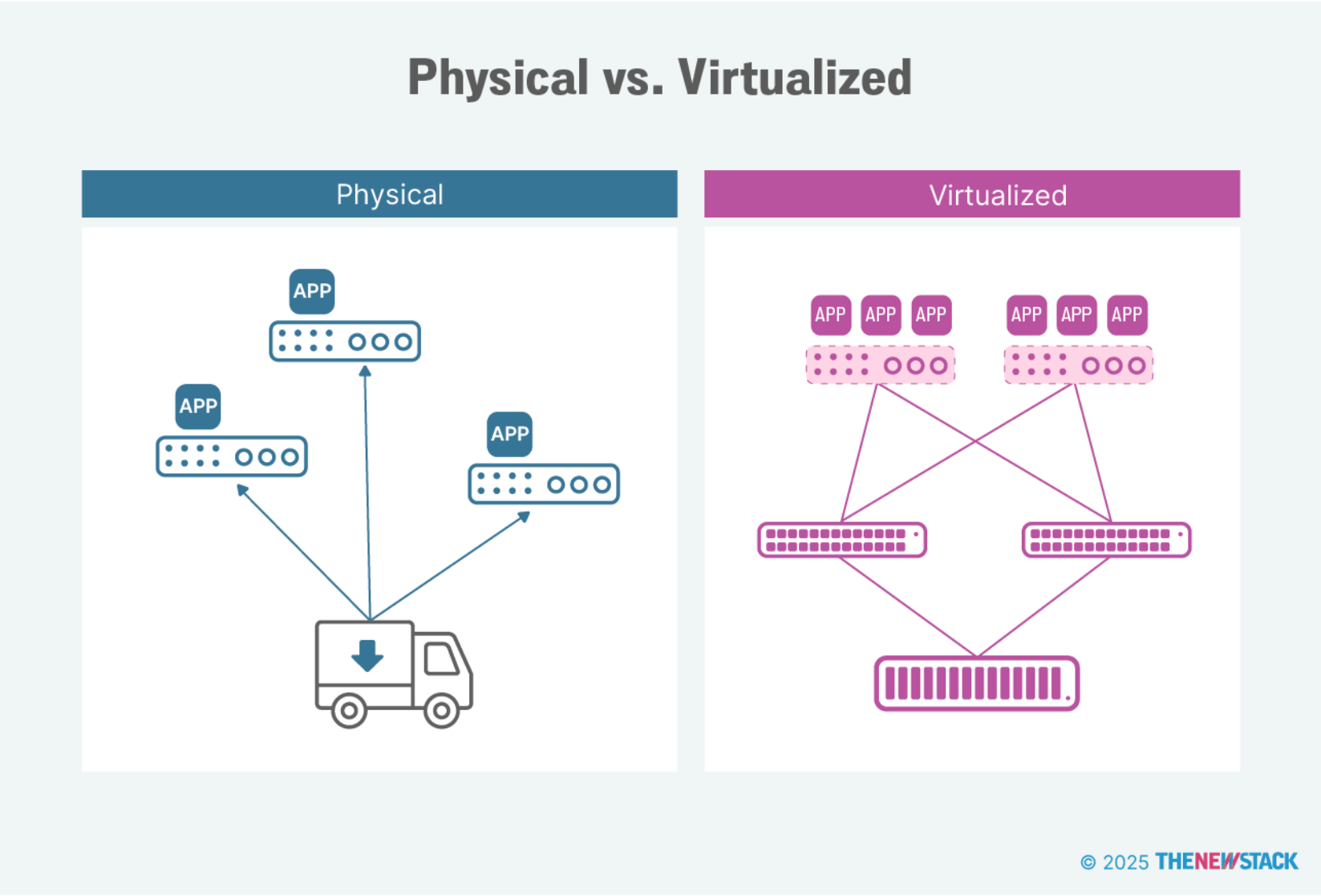


FIG 4.2 At small scale, either “rolling trucks” or virtualization will work for edge infrastructure deployment. But you need to plan ahead for Day 2 operations as you scale.

and two switches, with a single storage area network (SAN). This is a step up from sending trucks to various locations, but it is still very individualized — “more pets than cattle,” to use the old DevOps analogy.

Both approaches will work fine at a small scale. However, when planning any edge initiative, make sure to think ahead to your Day 2 operations — the ongoing management, update, and maintenance of clusters. You can’t avoid setting up physical sites with hardware and networking configuration at the start, but you don’t want servers to get out of sync if, for example, an update fails on one (i.e., you don’t want to create “snowflakes”). You must be able to deploy new applications across potentially thousands of sites without having to wait to roll trucks or deploy hardware.

Automation increases speed and agility, reduces errors, improves repeatability, enhances security, lowers costs, and provides higher reliability. By extending IaC practices to edge environments, DevOps teams can use automated,

version-controlled deployments for remote infrastructure. This, in turn, streamlines configuration and reduces deployment times.

GitOps has become a popular method for extending DevOps practices to edge environments. It uses git repositories as the single source of truth for both application and infrastructure configuration. When combined with the right platform, this approach provides:

- **Declarative configuration:** Describing desired state rather than imperative steps.
- **Version control:** Tracking all changes and enabling rollbacks.
- **Automated synchronization:** Ensuring edge sites match the desired state.
- **Audit trails:** Maintaining records of who changed what and when.

The Edge of Tomorrow

Using purpose-built tools specifically designed for running Kubernetes at the edge helps reduce complexity.

Likewise, selecting a Linux distribution specifically designed for edge scenarios can help mitigate many risks.

Options include:

- [**Bottlerocket**](#): Amazon's container-optimized Linux distribution.
- [**Chainguard OS**](#): A minimal, secure Linux distribution for containers.
- [**Flatcar**](#): A lightweight Linux distribution for containers.
- [**Kairos**](#): A cloud native Linux distribution for edge computing.
- [**Talos Linux**](#): A modern, secure, and minimal Linux distribution designed for Kubernetes.

These distributions are designed with security, minimalism, and automation in mind, reducing the attack surface and management overhead compared with traditional Linux distributions.

Successful edge computing requires more than just technical expertise — it demands a shift in our approach to distributed systems. By assembling diverse, collaborative teams, starting with low-risk workloads, embracing automation from the get-go, and choosing purpose-built tools, organizations can navigate the unique challenges of edge environments while unlocking their transformative potential. The edge represents not just the next frontier in our ongoing quest to reduce time to value, but a paradigm shift that brings computation closer to where decisions matter most.

As edge devices become increasingly capable and ubiquitous, the organizations that master cloud native practices at the edge will be best positioned to harness real-time insights, respond to local conditions, and deliver capabilities and experiences that were previously impossible. The future of enterprise computing is distributed, intelligent, and closer to the customer than ever before — and with the right approach, that future is within reach.

CONCLUSION

Final Words

Edge computing offers significant benefits, including enhancing privacy through federated learning, reducing carbon emissions by minimizing network traffic, eliminating cooling requirements, and lowering costs through cloud demigration. However, it also has distinct challenges: limited computing resources due to size, weight, and power constraints, as well as intermittent network connectivity that requires robust, fault-tolerant software design.

Containerization has become essential for edge deployments due to its lightweight nature and immutable infrastructure. Unlike traditional approaches requiring manual configuration of individual systems, containers enable predictable, repeatable deployments across thousands of edge locations. Container orchestration platforms like Kubernetes maintain familiar operational models while addressing edge-specific constraints.

Edge computing with Kubernetes represents a maturing technology addressing real business needs across industries. However, it requires specialized tools and approaches to manage the unique challenges of distributed, resource-constrained environments.

The combination of containerization, purpose-built operating systems, and centralized management platforms enables organizations including JYSK, PowerFlex, RapidAI, and Roche to deploy sophisticated applications at unprecedented scale while maintaining operational efficiency.

The emerging vendor ecosystem includes specialized solutions like Sidero's immutable, API-managed Talos Linux and Omni; Red Hat Device Edge, which extends OpenShift capabilities to edge locations; Spectro Cloud Palette, which offers comprehensive life cycle management; and Scale Computing HyperCore, which supports both VMs and containers. Each addresses different aspects of edge complexity while offering centralized management capabilities.

It's important to not overlook careful planning for Day 2 operations — the ongoing management and maintenance of distributed systems. Key strategies include starting with noncritical

workloads, implementing automation from the beginning, choosing purpose-built edge tools, and gradually increasing investment as confidence builds.

“Edge computing with Kubernetes represents a maturing technology addressing real business needs across industries.

Other Resources

With all of this in mind, I want to point you to other resources that may prove useful.

Unfortunately, the rapid change in Kubernetes at the edge means that the topic is quite poorly served in terms of books; several that I reviewed are already sufficiently out of date that I can't recommend them in good faith. However, "[The Edge Data Center](#)" by Hugh Taylor is a good introduction to the topic, suitable for managers and operations staff interested in edge computing. It provides a comprehensive introduction to 5G, smart devices, IoT, and smart cities, alongside useful information on edge data center design and modes of deployment.

Also, "[AI at the Edge: Solving Real-World Problems with Embedded Machine Learning](#)" is excellent. Authors Daniel Situnayake and Jenny Plunkett provide a practical, tightly written book that explains why bringing intelligence to edge devices is so important.

Two of my articles for The New Stack, which look at [federated learning](#) and questions around [AI and sustainability](#), would be good to read alongside this. For a more general introduction to sustainability, my previous ebook, "[The Developer's Guide to Cloud Infrastructure, Efficiency, and Sustainability](#)," is a good place to start.

Finally, though not specifically an edge book, "[Container Security](#)" by Liz Rice is a wonderful resource covering attack vectors that affect container deployments, measures for hardening containers, and how misconfigurations can compromise container isolation. A second edition is in the works and expected to be published in fall 2025.

About the Author



Charles Humble is an IT industry veteran with over 30 years of experience, currently working as a freelance consultant, author, and podcaster.

A former software engineer, architect, and CTO, he has worked as a senior leader and executive of both technology and content groups.

He was InfoQ's editor-in-chief from 2014 to 2020, and was chief editor for Container Solutions from 2020 to 2023.

He is the author of "[The Developer's Guide to Cloud Infrastructure, Efficiency, and Sustainability](#)" for The New Stack and "[Professional Skills for Software Engineers](#)" for O'Reilly, and hosts a regular series of [podcasts for GOTO on engineering leadership](#). He writes regularly for The New Stack and other publications, is a highly experienced content strategist, and has spoken at multiple international conferences including Devvxx, GOTO, and QCon. He is also a keyboard player and half of the ambient techno band [Twofish](#).

His primary areas of interest are how we build software better, including sustainability and ethics; cloud computing; remote working; diversity and inclusion; and inspiring the next generation of developers.

Disclosure

The following companies mentioned in this ebook are also sponsors of The New Stack: Amazon Web Services, Cloud Native Computing Foundation (CNCF), Red Hat, Spectro Cloud, and VMware by Broadcom.

The New Stack owner Insight Partners is also an investor in Docker.

